

# HÃY CHUẨN BỊ ĐÓN RẮC RỐI VÀ NHÂN ĐÔI NÓ!

## Xếp chồng học có giám sát - không giám sát cho phát hiện xâm nhập dựa trên bất thường

Tên gốc: “Prepare for Trouble and Make it Double! Supervised-Unsupervised Stacking for Anomaly-Based Intrusion Detection”

**Tommaso Zoppi, Andrea Ceccarelli**

Khoa Toán học và Tin học, Đại học Florence, Viale Morgagni 65, 50142 Florence, Ý

**Thông tin bài báo:** Chấp nhận tháng 4 năm 2021.

**Từ khóa:** Phát hiện xâm nhập; Tấn công zero-day; Phát hiện bất thường; Học có giám sát; Học không giám sát; Học máy; Stacking.

### TÓM TẮT

Trong những thập kỷ gần đây, các nhà nghiên cứu, chuyên gia thực hành và doanh nghiệp đã không ngừng tìm kiếm cơ chế phát hiện các hoạt động độc hại gây ra mối đe dọa an ninh. Trong số nhiều giải pháp, phát hiện xâm nhập mạng trở thành một trong những hướng phổ biến nhất để phân tích lưu lượng mạng và nhận biết các cuộc xâm nhập đang diễn ra, dựa trên luật hoặc dựa trên các mô hình học máy (ML) xử lý lưu lượng và học cách nghi ngờ một hành vi là xâm nhập. Các thuật toán ML có giám sát rất hiệu quả đối với những mối đe dọa đã biết, nhưng gặp khó khăn khi nhận diện tấn công zero-day - tức các cuộc tấn công không xuất hiện trong giai đoạn học - trong khi các thuật toán không giám sát có thể phát hiện loại tấn công này. Vì vậy, hai hướng có giám sát và không giám sát có các ưu điểm và nhược điểm bổ sung cho nhau. Tuy nhiên, vẫn chưa có câu trả lời dứt khoát về cách kết hợp hai hướng trong phát hiện xâm nhập mạng.

Bài báo trước hết làm rõ vấn đề tấn công zero-day và lý giải nhu cầu kết hợp thuật toán có giám sát với thuật toán không giám sát. Các tác giả đề xuất sử dụng học meta dưới dạng một Stacker hai tầng để tạo thành cách tiếp cận hỗn hợp có khả năng phát hiện cả mối đe dọa đã biết lẫn chưa biết. Sau đó, Stacker được hiện thực và đánh giá thực nghiệm thông qua một chiến dịch thí nghiệm nhằm: (i) thảo luận các đặc trưng meta được tạo bởi những bộ học không giám sát ở tầng cơ sở; (ii) lựa chọn các bộ phân loại có giám sát triển vọng nhất ở tầng meta; và (iii) đối sánh kết quả phân loại của Stacker với các bộ phân loại có giám sát và không giám sát. Cuối cùng, giải pháp được so sánh với các công trình gần đây. Nhìn chung, Stacker giảm số trường hợp phân loại sai so với các thuật toán ML có giám sát hoặc không giám sát trên cả 7 tập dữ liệu công khai được xem xét, đồng thời vượt các nghiên cứu trước trên 6 trong 7 tập dữ liệu. Đặc biệt, nó phát hiện tấn công zero-day hiệu quả hơn các thuật toán có giám sát, hạn chế điểm yếu chính của chúng mà vẫn duy trì khả năng phù hợp đối với các cuộc tấn công đã biết.

## 1. GIỚI THIỆU

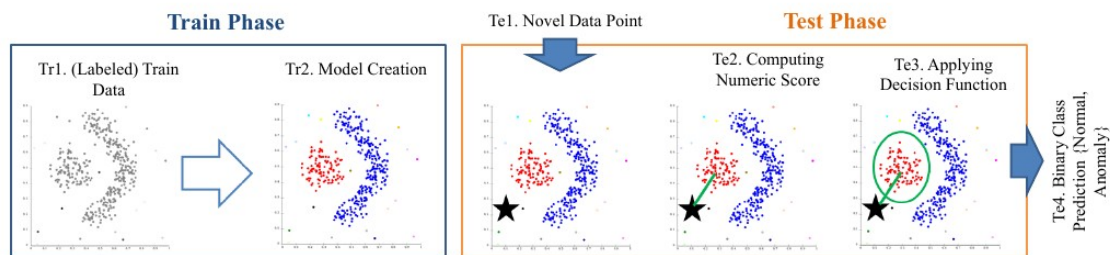
Ngày nay, các hệ thống công nghệ thông tin và truyền thông (ICT) thường tích hợp những chức năng được thực hiện, điều khiển hoặc giám sát một phần bởi các phân hệ phân tán. Các mô hình như hệ thống không gian mạng - vật lý (CPS), hệ thống đám mây và điện toán cụm thường kết nối nhiều thành phần hoặc phân hệ để cung cấp các chức năng đó. Vì thường cung cấp quyền truy cập từ xa tới dịch vụ và thông tin quan trọng, chúng có thể trở thành mục tiêu của các cuộc tấn công mạng [2], [6], [92], [95]. Trong thập kỷ vừa qua, tác động của các mối đe dọa mạng liên tục gia tăng như được chỉ ra trong nhiều báo cáo [7], [8]. Do đó, hệ thống phát hiện xâm nhập (IDS) [3], [4] đã trở thành thành phần thiết yếu để phát hiện mối đe dọa tiềm ẩn và kích hoạt các mô-đun chặn hoặc giảm nhẹ tác động bất lợi của chúng. IDS thu thập và phân tích dữ liệu mạng, và thường cả các chỉ báo hệ thống, để phát hiện hoạt động độc hại hoặc trái phép dựa trên giả thuyết rằng một cuộc tấn công đang diễn ra sẽ tạo ra những ảnh hưởng có thể phân biệt trên các chỉ báo này.

## 1.1. Chữ ký và bất thường

Dữ liệu mạng và các chỉ báo hệ thống đã được sử dụng nhiều thập kỷ làm cơ sở xây dựng chữ ký của những cuộc tấn công đã biết. Chẳng hạn, các trường tiêu đề gói tin hoặc thư điện tử bị định dạng sai, có thể gây cạn kiệt tài nguyên [9], có thể được phát hiện bằng cách kiểm tra miền giá trị của các trường trong gói tin đến. Tương tự, mã độc tổng tiền có thể được phát hiện hiệu quả [10] bằng cách phân tích lời gọi hệ thống để xem chúng có thực hiện các hàm mật mã hay không. Các cách tiếp cận dựa trên chữ ký hoạt động tốt đối với tấn công đã biết [2], [3], [4], nhưng yếu trong việc phát hiện biến thể của tấn công đã biết hoặc tấn công zero-day [6], vì chữ ký của chúng chưa tồn tại.

Phương án thay thế là phát hiện xâm nhập dựa trên bất thường [1]. Hoạt động đáng ngờ có thể được phát hiện bằng thuật toán học máy dựa trên các quan sát mạng và hệ thống trong quá khứ để học một mô hình, rồi phân loại các điểm dữ liệu mới thành bình thường hoặc bất thường, tức có khả năng là tấn công. Hình 1 minh họa một bộ phân loại nhị phân trước hết được huấn luyện (Tr1, Tr2) để học mô hình. Giai đoạn huấn luyện chủ yếu dùng nhãn cho biết điểm dữ liệu có tương ứng với tấn công hay không. Sau đó, bộ phân loại dự đoán nhãn của một điểm dữ liệu mới (Te1) bằng thuật toán ML tạo ra một giá trị số, chẳng hạn khoảng cách đến tâm của cụm gần nhất (Te2). Giá trị này được đưa vào hàm quyết định, ví dụ: “điểm dữ liệu có đủ gần tâm cụm để có thể xem là thuộc cụm hay không?” (Te3). Hàm quyết định trả về nhãn nhị phân (Te4) xác định điểm dữ liệu có bất thường hay không. Hàm quyết định có thể được tích hợp trong bộ phân loại hoặc là tham số của bộ phân loại. Trong ví dụ phân cụm, nó có thể là: (i) một ngưỡng tĩnh; (ii) một ngưỡng phụ thuộc vào độ thưa của cụm; hoặc (iii) một ngưỡng liên quan đến số phần tử trong cụm.

Figure 1: Main Steps for Train (Tr1, Tr2) and Test (Te1, Te2, Te3, Te4) phases of Binary Classifiers.



Hình 1

**Hình 1.** Các bước chính trong giai đoạn huấn luyện (Tr1, Tr2) và kiểm thử (Te1, Te2, Te3, Te4) của bộ phân loại nhị phân. Nội dung nhãn trong hình: Train Phase = Giai đoạn huấn luyện; Test Phase = Giai đoạn kiểm thử; Tr1 = Dữ liệu huấn luyện có nhãn; Tr2 = Tạo mô hình; Te1 = Điểm dữ liệu mới; Te2 = Tính điểm số; Te3 = Áp dụng hàm quyết định; Te4 = Dự đoán lớp nhị phân {Bình thường, Bất thường}.

## 1.2. Phát hiện xâm nhập có giám sát và không giám sát

Các bộ phát hiện xâm nhập dựa trên ML có giám sát cần những quan sát lịch sử đã biết nhãn, hay còn gọi là lớp. Chúng học một mô hình để phân loại mỗi quan sát mới - tức điểm dữ liệu - là được thu thập: (i) khi hệ thống đang chịu tấn công; hoặc (ii) trong quá trình vận hành bình thường. Nhiều nghiên cứu cho thấy Random Forest [31], máy vector hỗ trợ (SVM) [33], mạng nơ-ron tích chập [34] và mạng nơ-ron sâu [35], [38] có thể phát hiện mối đe dọa bằng cách phân tích lưu lượng mạng. Các thuật toán có giám sát này rất hiệu quả với mối đe dọa đã biết, nhưng không xử lý tốt mối đe dọa mới hoặc chưa biết. Đây là nhược điểm nghiêm trọng vì nhiều mối đe dọa không thể được xác định ngay từ giai đoạn thiết kế hệ thống.

Ngược lại, thuật toán ML không giám sát không giả định có sẵn kiến thức về tấn công. Chúng mô hình hóa hành vi kỳ vọng, tức hành vi bình thường của hệ thống, và coi mọi sai lệch khỏi hành vi bình thường là bất thường, tức một cuộc tấn công bị nghi ngờ [1]. Thuật toán phân cụm [65], [69] có lẽ là nhóm được sử dụng rộng rãi nhất, bên cạnh các phương pháp thống kê [60], dựa trên góc [46], dựa trên mật độ [70], [63], các biến thể không giám sát của mạng nơ-ron [64], phương pháp dựa trên láng giềng [61], [71] hoặc dựa trên phân loại [72], [62] [29], [30]. Vì mô hình được xây dựng không dựa vào nhãn, thuật toán không giám sát không phân biệt tấn công đã biết với tấn công chưa biết/zero-day. Tuy nhiên, chúng dễ phân loại sai các cuộc tấn công đã biết hơn thuật toán có giám sát.

### 1.3. Tính bổ sung và việc kết hợp hai nhóm thuật toán

Như phân tích trên, hai hướng có giám sát và không giám sát bổ sung cho nhau.

Kỹ thuật có giám sát học từ các đợt tấn công trong quá khứ và chủ yếu mô hình hóa hành vi độc hại đã biết. Vì vậy, sử dụng bộ phân loại có giám sát luôn đi kèm điểm yếu cố hữu đối với mối đe dọa zero-day hoặc những biến đổi nhỏ của tấn công đã biết. Dù có thể áp dụng kỹ thuật chống quá khớp, chẳng hạn cắt tỉa cây quyết định [5], để tránh xây dựng mô hình quá bám sát một cuộc tấn công cụ thể, không có cách đáng tin cậy nào biến bộ phân loại có giám sát thành giải pháp phù hợp để phát hiện mối đe dọa chưa từng biết. Trong khi đó, kỹ thuật không giám sát mô hình hóa hành vi bình thường và coi mọi độ lệch là bất thường.

Mỗi hướng có thể mạnh và hạn chế riêng nên hiệu quả thay đổi theo kịch bản. Vì vậy, việc ghép bộ phân loại có giám sát với bộ phân loại không giám sát là rất đáng mong muốn, như trong [12], [13], [14], [17]. Tuy nhiên, phần lớn giải pháp này phụ thuộc mạnh vào bài toán cụ thể và khó khái quát. Hơn nữa, kết hợp hai hướng không hề đơn giản và không phải lúc nào cũng cải thiện khả năng phát hiện: một số thuật toán gây nhiễu có thể kéo tổ hợp về phía quyết định sai, tạo tác động bất lợi rõ ràng.

Một chiến lược tiềm năng đến từ lĩnh vực học meta [74]. Bộ học meta, hay bộ phân loại meta, điều phối một tập hợp thuật toán ML để nâng cao khả năng phân loại. Chẳng hạn, Random Forest [31] và AdaBoost [32] lần lượt tạo các tổ hợp đồng nhất theo Bagging [21] và Boosting [22] từ cây quyết định. Voting [36] được dùng rộng rãi trong thiết kế hệ thống dư thừa [37]. Các hướng khác như Stacking [23], Cascading [24], Arbitrating [27] và Delegating [26] ít phổ biến hơn nhưng vẫn có ý nghĩa [28].

### 1.4. Đóng góp và cấu trúc bài báo

Theo hiểu biết của các tác giả, tài liệu hiện có chưa đạt đồng thuận rõ ràng và cũng chưa có xác nhận thực nghiệm về cách điều phối, kết hợp đúng đắn bộ phân loại có giám sát với không giám sát để phát hiện xâm nhập mạng, dù đã có nhiều nỗ lực. Vì vậy, bài báo trước hết tổng quan các chiến lược kết hợp bộ phân loại cho phát hiện tấn công (Mục 2) và đánh giá tính phù hợp của học meta trong việc kết hợp hai nhóm bộ phân loại (Mục 3). Từ đó, Mục 4 đề xuất cách tiếp cận Stacking gồm nhiều bộ học không giám sát ở tầng cơ sở và một bộ học có giám sát ở tầng meta nhằm cải thiện khả năng phát hiện cả mối đe dọa đã biết lẫn chưa biết/zero-day.

Mục 5 thiết kế, thiết lập và thực hiện chiến dịch thí nghiệm trên các tập dữ liệu tấn công công khai và các thuật toán có giám sát/không giám sát phổ biến để làm rõ lợi ích của Stacker so với thuật toán hiện đại. Mục 6 phân tích cách các bộ học cơ sở tạo ra những đặc trưng meta bổ sung và có giá trị [75], sau đó thảo luận hiệu năng phân loại của phương pháp so với thuật toán có giám sát và không giám sát. Kết quả cho thấy Stacker phân loại tấn công đã biết tốt hơn thuật toán có giám sát, đồng thời cải thiện rõ rệt phát hiện tấn công zero-day. Mục 7 trình bày cách xây dựng Stacker cho một hệ thống đích; Mục 8 so sánh kết quả với các nghiên cứu đã công bố chỉ số định lượng về phát hiện xâm nhập; và Mục 9 kết luận, nêu hướng nghiên cứu tương lai.

## 2. CÁC CÔNG TRÌNH TRƯỚC VỀ KẾT HỢP BỘ PHÂN LOẠI CÓ GIÁM SÁT VÀ KHÔNG GIÁM SÁT

Trong [12], [79], các tác giả kết hợp chiến lược không giám sát - cụ thể là phân cụm - với bộ phân loại có giám sát: các đặc trưng bổ sung phụ thuộc vào cụm được đưa cho bộ phân loại có giám sát, và bộ này đưa ra quyết định cuối. Kết quả đáng khích lệ, nhưng họ chỉ ra một vấn đề quan trọng: “bổ sung quá nhiều điểm số vào tập dữ liệu có thể gây hại do quá khớp và vấn đề phương sai” [12]. Nói cách khác, dữ liệu thực nghiệm cho thấy kiến thức thu được từ nhiệm vụ không giám sát có thể trở thành nhiễu và cản trở thay vì hỗ trợ quá trình học có giám sát. Tương tự, trong [15], các tác giả xử lý KDDCup99 bằng một Stacker chạy mô hình xác suất không giám sát trước bộ sinh luật có giám sát. Họ kết luận cách tiếp cận hỗ trợ phát hiện mẫu

chung của một nhóm ngoại lệ, nhưng kết quả chỉ đúng với KDDCup99 - một tập dữ liệu hiện đã lỗi thời - và chưa được lặp lại trên dữ liệu cùng kiểu tấn công hiện đại.

Một nghiên cứu khác [14] đặt mục tiêu “phát hiện và cô lập luồng độc hại khỏi lưu lượng mạng, rồi tiếp tục phân loại thành một loại mã độc đã biết, biến thể của mã độc đã biết hoặc một chủng mã độc hoàn toàn mới”. Trước hết, họ thiết lập bộ phân loại nhị phân để phân biệt luồng dữ liệu bình thường với bất thường; sau đó, một bộ phân loại kết hợp có giám sát - không giám sát tách các lớp mã độc cụ thể. Mã độc cũng là đối tượng của [13], trong đó một bộ phát hiện mã độc Android lại phân tích cả quyền truy cập và đặc trưng lưu lượng. Tương tự [12], nghiên cứu này chạy phân cụm trước rồi xây dựng bộ phân loại k láng giềng gần nhất (kNN), cho kết quả tốt hơn kNN chỉ dùng đặc trưng gốc của tập dữ liệu. Trong [14], các tầng được dùng để nhận dạng đa lớp mã độc đã phát hiện; còn [13] và [14] dùng phân cụm để lọc sơ bộ dữ liệu trước phân loại có giám sát.

Đáng chú ý, bộ học kết hợp có giám sát - không giám sát cũng được đề xuất ngoài lĩnh vực an ninh. Chúng được dùng trong vật lý để dự đoán bùng phát Mặt Trời [18] hoặc cải thiện autoencoder [19], gần đây còn hỗ trợ phân loại y tế [17] và xử lý ảnh [16]. Một số nghiên cứu chạy song song các bộ phân loại dị thể rồi hợp nhất kết quả theo luật tính, ví dụ voting [36], vốn hữu ích cho hệ thống dư thừa [37] và hiện được dùng trong nhiều miền.

### 3. HỌC META ĐỂ KẾT HỢP BỘ PHÂN LOẠI CÓ GIÁM SÁT VÀ KHÔNG GIÁM SÁT

Việc kết hợp hai nhóm bộ phân loại thường dẫn tới một quá trình phân loại nhiều tầng, trong đó kết quả của từng bộ phân loại đóng góp vào đầu ra cuối và có thể cung cấp thông tin bổ sung cho các bộ phân loại khác.

#### 3.1. Học meta để kết hợp mô hình

Bài báo trước hết thảo luận các chiến lược đang được dùng để kết hợp tập hợp bộ phân loại đồng nhất và dị thể. Theo [74], bộ học meta là bộ phân loại sử dụng tri thức thu được trong các phiên học cơ sở - tức tri thức meta - để cải thiện khả năng phân loại ở tầng meta. Cụ thể hơn, quá trình học cơ sở đưa đặc trưng của tập dữ liệu vào một hoặc nhiều thuật toán học để tạo ra một hay nhiều mô hình dùng ở tầng cơ sở. Kết quả của các bộ học cơ sở góp phần tạo các đặc trưng meta [75], sau đó được đưa cùng các đặc trưng khác vào bộ phân loại tầng meta để tính đầu ra của toàn bộ bộ học meta.

Các loại bộ học meta chính gồm:

- **Bagging [21]:** kết hợp các bộ học cơ sở cùng loại bằng nhiều bản sao bootstrap của tập huấn luyện. Kết quả tổ hợp được xác định theo đa số đầu ra của các bộ học cơ sở.
- **Boosting [22]:** xây dựng một tổ hợp các bộ học yếu đồng nhất. Một bộ học yếu có thể không đạt hiệu năng cao, nhưng Boosting điều phối nhiều bộ học yếu để hình thành bộ học meta mạnh.
- **Voting có hoặc không trọng số [36]:** đếm ý kiến của một tập hợp dị thể và đưa ra quyết định cuối bằng phép tổng hợp tuyến tính. Theo luật  $k$  trên  $N$ , nếu ít nhất  $k$  trong  $N$  thành viên đồng thuận thì kết quả đó được chọn.
- **Stacking [23]:** sử dụng các thuật toán khác nhau, được huấn luyện trên cùng một tập huấn luyện, làm bộ học cơ sở. Đầu ra của chúng trở thành đặc trưng meta dựa trên mô hình và được đưa vào một bộ phân loại độc lập ở tầng meta. Khác Bagging và Boosting, kết quả cuối không dựa trên biểu quyết đa số mà do bộ phân loại độc lập kết hợp các đầu ra bằng một hàm tổng quát, có thể phi tuyến.
- **Cascading [24]:** phát triển từ Boosting nhưng dùng các bộ học yếu dị thể. Điểm dữ liệu lần lượt đi qua chuỗi bộ phân loại; bộ phân loại đầu tiên đủ tự tin sẽ tạo đầu ra cuối.

- **Delegating [26]:** tương tự Cascading, nhưng mọi bộ học cơ sở được huấn luyện trên cùng phần dữ liệu. Phương pháp thận trọng này chuyển quyền quyết định cho bộ học cơ sở tiếp theo nếu bộ phân loại hiện tại không đủ tự tin.
- **Cascade Generalization [25]:** kết hợp tuần tự các bộ phân loại dị thể. Thay vì chạy song song như Stacking, bộ phân loại thứ  $i+1$  đóng vai trò bộ học meta cho bộ thứ  $i$ ; mỗi bộ phân loại vừa là bộ học cơ sở vừa là bộ học meta. Tương tự Cascading, bộ đầu tiên đủ tự tin sẽ tạo đầu ra.

### 3.2. Học meta để kết hợp học có giám sát và không giám sát

Bộ học meta đã được dùng thành công để cải thiện phân loại có giám sát [31], [32] và gần đây là học không giám sát [28]. Tuy nhiên, việc dùng chúng để kết hợp hai nhóm không phải điều hiển nhiên và cần được phân tích cẩn thận.

Bagging và Boosting là tổ hợp đồng nhất, do đó không cho phép đồng thời dùng bộ phân loại có giám sát và không giám sát. Vì huấn luyện bộ học yếu trong Boosting dựa trên việc nhận diện các vùng khó phân loại của tập huấn luyện, với giả định nhãn huấn luyện đã biết, có thể nghĩ tới việc dùng bộ học cơ sở không giám sát để xây dựng Boosting. Theo [28], giải pháp này tốt hơn phần lớn bộ phân loại không giám sát, nhưng vẫn tạo nhiều phân loại sai hơn đáng kể so với bộ phân loại có giám sát khi xử lý tấn công đã biết.

Cascading, Delegating và Cascade Generalization có tiềm năng vì tích hợp các bộ phân loại dị thể, nhưng phụ thuộc mạnh vào khái niệm độ tin cậy của dự đoán. Giả sử thuật toán trả xác suất cho hai lớp {bình thường, bất thường} và chọn lớp có xác suất cao hơn; xác suất thuộc lớp chiếm ưu thế càng cao thì độ tin cậy càng lớn. Tuy nhiên, không phải thuật toán nào cũng gán xác suất lớp rõ ràng. Độ tin cậy còn có thể phụ thuộc vào hàm quyết định dùng để đổi điểm số thành nhãn nhị phân (Hình 1, bước Te3), và cách tính thay đổi giữa các hiện thực. Vì vậy, bộ học meta dựa trên độ tin cậy có thể gặp khó khăn khi thiếu một cách tính chuẩn hóa, đặc biệt khi kết hợp nhiều thuật toán có giám sát/không giám sát với nhiều hàm quyết định, chẳng hạn khoảng tứ phân vị [57].

Stacking có thể phần nào tránh vấn đề ngưỡng và độ tin cậy vì dùng trực tiếp đầu ra số của bộ học cơ sở làm đặc trưng dựa trên mô hình rồi đưa vào bộ phân loại tầng meta [76], [83]. Bộ phân loại tầng meta chịu trách nhiệm tổng hợp và tạo kết quả thống nhất mà không cần tính độ tin cậy. Vì vậy, việc chọn bộ phân loại tầng meta đóng vai trò quan trọng và phải được lập kế hoạch cẩn thận.

Voting có trọng số cần được xem xét riêng. Dù được dùng rộng rãi để quản lý các thành phần dư thừa và thường dị thể [37], [36], nó khó thích nghi để bỏ phiếu trên quyết định của cả bộ phân loại có giám sát lẫn không giám sát. Giả sử ba thuật toán có giám sát  $S_1$ ,  $S_2$ ,  $S_3$  và hai thuật toán không giám sát  $U_1$ ,  $U_2$  chạy song song, sau đó bỏ phiếu theo luật  $k$  trên 5. Làm thế nào chọn  $k$  mà không thiên lệch về phía một nhóm? Khi tấn công zero-day xảy ra, các thuật toán có giám sát có thể gán “bình thường”, trong khi  $U_1$  và  $U_2$  có khả năng gán “bất thường”. Vì vậy, đặt ngưỡng như  $k = 3$  mà không xét bộ nào phát cảnh báo là không khả thi. Voting có trọng số cũng có thể làm tăng: (i) dương tính giả nếu đặt trọng số lớn hơn cho thuật toán không giám sát; hoặc (ii) âm tính giả nếu ưu tiên  $S_1$ ,  $S_2$  và  $S_3$ , khiến tấn công zero-day bị phân loại sai và không được phát hiện.

## 4. STACKING CÓ GIÁM SÁT - KHÔNG GIÁM SÁT

Tóm lại, Stacking là kỹ thuật học meta triển vọng nhất để kết hợp mô hình có giám sát và không giám sát cho phát hiện xâm nhập. Mục này trình bày các đặc điểm chính của Stacking và cách hiện thực một Stacker phối hợp hai loại bộ phát hiện.

### 4.1. Lựa chọn bộ học cơ sở

Stacking dựa trên nhiều bộ học cơ sở khác nhau, được xây dựng bằng cùng đặc trưng tập dữ liệu nhưng dùng các thuật toán ML dị thể. Đầu ra của chúng tạo ra đặc trưng meta dựa trên mô hình [75]; những đặc

trường này, cùng đặc trưng gốc của tập dữ liệu, được đưa vào bộ phân loại độc lập ở tầng meta để tạo kết quả cuối. Các đặc trưng tập dữ liệu vẫn cần được đưa cho bộ học meta bên cạnh đặc trưng dựa trên mô hình vì chúng mô tả trực tiếp hệ thống.

Việc chọn đặc trưng meta, và do đó chọn bộ phân loại tầng cơ sở để tạo chúng, có vai trò quyết định. Có thể dùng ba cách kết hợp sau; theo lập luận ở Mục 4.1.2, bài báo xem bộ học cơ sở không giám sát là phương án có lợi nhất.

#### 4.1.1. Bộ học cơ sở có giám sát

Nếu các bộ học cơ sở là có giám sát, cần dùng bộ phân loại tầng meta không giám sát để bảo đảm hai hướng đều hiện diện. Tuy nhiên, các mô hình không giám sát có thể phân loại sai tấn công đã biết [2], [20], [30], làm tăng tổng số lỗi và khiến toàn bộ quá trình học meta nghiêng về cách tiếp cận không giám sát truyền thống.

#### 4.1.2. Bộ học cơ sở không giám sát

Ngược lại, đưa đặc trưng dựa trên mô hình không giám sát vào bộ phân loại tầng meta có giám sát có thể cân bằng hai hướng. Bộ phân loại có giám sát học mối liên hệ giữa giá trị đặc trưng hoặc tổ hợp điều kiện cụ thể với nhãn bất thường, nên yếu trước những kiểu chưa biết vì chúng có thể làm thay đổi đặc trưng theo cách không dự đoán được. Nếu đặc trưng meta được sinh bởi các bộ học không giám sát, đầu ra của tầng meta có giám sát sẽ phụ thuộc trực tiếp không chỉ vào đặc trưng gốc mà còn vào các đặc trưng dựa trên mô hình này, nhờ đó có thể đạt độ chính xác cao hơn với tấn công zero-day.

#### 4.1.3. Kết hợp cả hai loại bộ học cơ sở

Phương án cuối dùng đồng thời bộ học có giám sát và không giám sát ở tầng cơ sở. Ưu điểm rõ ràng là tách lựa chọn bộ học tầng meta khỏi lựa chọn bộ học cơ sở: bất kể tầng meta dùng loại nào, cả hai hướng vẫn xuất hiện ở tầng cơ sở. Tuy nhiên, cấu hình này sẽ làm nổi trội một phía - học có giám sát nếu bộ phân loại tầng meta có giám sát, hoặc học không giám sát nếu ngược lại - và khó tránh khỏi làm lệch cân bằng giữa hai hướng.

### 4.2. Xác định đặc trưng meta dựa trên mô hình

#### 4.2.1. Điểm số và nhãn nhị phân của thuật toán

Bài báo khảo sát các đặc trưng meta tiềm năng cho Stacker. Như Hình 1, giả sử mỗi thuật toán không giám sát ua tính cả điểm số ua.num và nhãn nhị phân ua.bin, trong đó ua.bin thu được sau khi áp dụng hàm quyết định lên ua.num (các bước Te2 và Te4).

Cả hai đều quan trọng: ua.bin là lớp cuối cùng do bộ phân loại dự đoán sau hàm quyết định; còn ua.num là một giá trị số thuần túy do thuật toán gán, chưa qua xử lý bổ sung.

#### 4.2.2. Đầu ra từ nhiều thuật toán không giám sát đa dạng

Việc dùng đồng thời nhiều hơn một bộ học cơ sở không giám sát để tạo đặc trưng meta được xem là có lợi. Gọi  $UA = \{ua_1, ua_2, \dots, ua_k\}$  là tập gồm  $uk$  thuật toán không giám sát. Tập này tạo tổng cộng  $2 \times uk$  đặc trưng meta, tức  $uai.num$  và  $uai.bin$  cho mỗi  $uai$ .

Không thể xác định  $UA$  nếu không biết: (i) khả năng phát hiện của từng thuật toán; và (ii) tính hiệp lực giữa chúng. Các nghiên cứu trước cho thấy dùng thành phần đa dạng [55] giúp tránh lỗi chế độ chung [56], gợi ý chọn nhiều thuật toán khác nhau để giảm phân loại sai. Tuy nhiên, xác định hai thuật toán không giám sát có thật sự đa dạng hay không không hề đơn giản. Các thuật toán có thể khác nhau về cách phân loại điểm dữ liệu nhưng vẫn dùng cùng một heuristic. Ví dụ, ODIN [61], FastABOD [46], LOF [70] và COF [67] đều thực hiện tìm kiếm kNN để tính điểm hoặc giảm chi phí tính toán. Chúng khác nhau nhưng không hoàn toàn

đa dạng vì cùng dựa trên khái niệm láng giềng. Trong nghiên cứu này, hai thuật toán được xem là đa dạng nếu không thuộc cùng một họ: phân cụm, mật độ, mạng nơ-ron, góc, thống kê, phân loại hoặc láng giềng.

#### 4.2.3. Bộ đếm biểu quyết làm đặc trưng meta

Voting cũng có thể tạo thêm đặc trưng meta. Một bộ bỏ phiếu có hoặc không trọng số trước hết tính bộ đếm, sau đó áp dụng ngưỡng được học trong huấn luyện để phân loại nhị phân. Để tạo đặc trưng meta, chỉ cần tính bộ đếm vì ngưỡng có thể được học bởi bộ phân loại tầng meta có giám sát.

Với tập  $UA$  sinh ra đặc trưng num và bin, hai bộ đếm được định nghĩa:

$$Count(UA) = \sum_{ua_i \in UA} ua_i.bin$$

$$WCount(UA, rep) = \frac{1}{|UA|} \sum_{ua \in UA} \frac{1}{2} [1 - (-1)^{ua.bin \times rep(ua)}], -1 \leq rep(ua) \leq 1$$

$Count(UA)$  đơn giản là số thuật toán  $ua$  trong  $UA$  có  $ua.bin = 1$ , tức số bộ phân loại không giám sát coi điểm dữ liệu là bất thường; vì vậy  $0 \leq Count(UA) \leq |UA|$ .  $WCount(UA, rep)$  là bộ đếm có trọng số, mở rộng  $Count$  bằng khái niệm uy tín. Mỗi thuật toán  $ua$  được gán  $rep(ua)$  trong  $[-1, 1]$ , trong đó -1 là thuật toán luôn sai, 1 là luôn đúng và 0 tương đương đoán ngẫu nhiên. Giá trị  $WCount$  dao động từ 0 - rất tin rằng điểm dữ liệu là bình thường - đến 1, chỉ đạt được khi tất cả thuật toán có uy tín tối đa và cùng đồng thuận rằng điểm dữ liệu là bất thường.

Figure 2: Example of Dataset Features (green solid box), Algorithms Features (red dashed box) and voting counters (others).

| packets | bytes | Duration | AlgA.num | AlgB.num | AlgA.bin | AlgB.bin | Count | WCount | label   |
|---------|-------|----------|----------|----------|----------|----------|-------|--------|---------|
| 13      | 13005 | 0        | 0.02     | 20552    | 1        | 1        | 2     | 0.775  | anomaly |
| 11      | 1292  | 0        | 0.17     | 1811     | 1        | 1        | 2     | 0.775  | anomaly |
| 271     | 13193 | 13       | 0.02     | 20857    | 1        | 1        | 2     | 0.775  | anomaly |
| 1       | 142   | 0        | 0.50     | 29       | 1        | 0        | 1     | 0.375  | normal  |
| 1       | 160   | 0        | 4.78     | 0        | 0        | 0        | 0     | 0.225  | normal  |

Hình 2

**Hình 2.** Ví dụ về đặc trưng tập dữ liệu (khung xanh liền), đặc trưng thuật toán (khung đỏ nét đứt) và các bộ đếm biểu quyết. Các cột lần lượt là: số gói tin, số byte, thời lượng, điểm số của AlgA và AlgB, nhãn nhị phân của AlgA và AlgB, Count, WCount và nhãn cuối. Ví dụ dùng  $UA = \{AlgA, AlgB\}$ ,  $rep(AlgA)=0,2$ ,  $rep(AlgB)=0,8$ .

### 4.3. Lựa chọn bộ phân loại tầng meta

Các bộ học cơ sở không giám sát tạo ra đặc trưng dựa trên mô hình để bộ phân loại tầng meta có giám sát phát hiện tấn công. Bộ phân loại này nên được chọn trong số những thuật toán đã chứng minh hiệu quả. Vì bộ phân loại nhị phân có giám sát đã được dùng nhiều thập kỷ, có rất nhiều lựa chọn. Nghiên cứu không đặt ràng buộc cho tầng meta và khảo sát nhiều họ: dựa trên láng giềng [39], Boosting [32], cây quyết định và tổ hợp cây [31], SVM [33], phương pháp thống kê [40] và mạng nơ-ron sâu có embedding [59].

## 5. THIẾT LẬP THỰC NGHIỆM

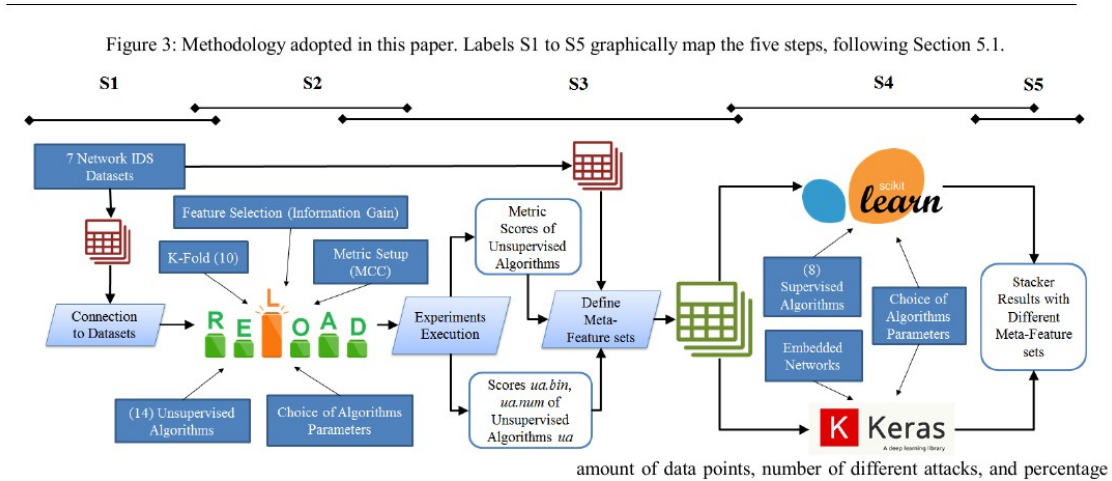
### 5.1. Phương pháp

Để xây dựng và đánh giá Stacker, các tác giả thực hiện một chiến dịch thí nghiệm gồm năm bước S1-S5, được minh họa trong Hình 3.

- **S1.** Thu thập các tập dữ liệu tấn công công khai chứa dữ liệu mạng của hệ thống thực hoặc mô phỏng, đồng thời tạo các biến thể cho phép phân tích tấn công chưa biết (Mục 5.2).

- **S2.** Tổng quan tài liệu để chọn: (i) các thuật toán không giám sát phù hợp cho phát hiện bất thường (Mục 5.3); và (ii) các chỉ số đánh giá (Mục 5.4).
- **S3.** Áp dụng từng thuật toán không giám sát lên từng tập dữ liệu để tạo đặc trưng meta dựa trên mô hình và thu thập giá trị chỉ số; từ đó xác định các tập đặc trưng meta (Mục 5.5).
- **S4.** Xác định từ tài liệu các bộ phân loại nhị phân có giám sát sẽ được dùng làm bộ học tăng meta trên những đặc trưng được tính ở S3 (Mục 5.6).
- **S5.** Thực hiện thí nghiệm theo thiết lập trình bày tại Mục 5.7.

Kết quả của S5 được phân tích tại Mục 6, cung cấp nhiều góc nhìn, đưa ra bằng chứng về ưu thế của Stacking và xác định cấu hình được khuyến nghị cho Stacker có giám sát - không giám sát.



Hình 3

**Hình 3.** Phương pháp được sử dụng trong bài báo. Các nhãn S1-S5 tương ứng năm bước ở Mục 5.1: chọn tập dữ liệu và tạo biến thể zero-day; chọn thuật toán không giám sát và chỉ số; sinh đặc trưng meta; chọn bộ phân loại có giám sát tăng meta; thực hiện và phân tích thí nghiệm.

## 5.2. Các tập dữ liệu công khai được lựa chọn

Mục này trình bày bảy tập dữ liệu công khai dùng trong nghiên cứu. Bảng 1 tổng hợp tên tập dữ liệu, kích thước phần dữ liệu được chọn, số đặc trưng thứ tự, số nhóm tấn công, tỷ lệ bản ghi tấn công và năm phát hành.

**Bảng 1.** Các tập dữ liệu dùng trong nghiên cứu

| Tập dữ liệu  | Số điểm dữ liệu | Số đặc trưng thứ tự | Số nhóm tấn công | Tỷ lệ tấn công (%) | Năm  |
|--------------|-----------------|---------------------|------------------|--------------------|------|
| ADFA-Netflow | 132.002         | 3                   | 5                | 31,03              | 2017 |
| CIDD5-001    | 400.000         | 7                   | 4                | 39,16              | 2017 |
| ISCX12       | 400.000         | 6                   | 4                | 7,96               | 2012 |
| NSL-KDD      | 148.516         | 37                  | 4                | 48,12              | 2009 |
| UGR16        | 207.256         | 7                   | 5                | 12,99              | 2016 |
| CICIDS17     | 500.000         | 75                  | 4                | 29,88              | 2017 |
| UNSW-NB15    | 175.341         | 38                  | 7                | 38,06              | 2015 |

### Các tập dữ liệu được chọn

Bắt đầu từ các khảo sát gần đây [41], [90] và việc tra cứu các cổng dữ liệu trực tuyến, nhóm tác giả lựa chọn những tập dữ liệu đáp ứng: (i) được công bố trong mười năm gần nhất; (ii) có nhãn ít nhất một phần; (iii) có tối thiểu 100.000 điểm dữ liệu để bảo đảm quy mô đủ lớn; và (iv) từng được dùng cho phát hiện xâm nhập dựa trên bất thường, nhằm tạo điều kiện so sánh kết quả.

Quá trình lựa chọn thu được các tập dữ liệu sau: ISCX12 - dùng một phần nhật ký Chủ nhật, Thứ hai, Thứ ba và Thứ năm để bao gồm bốn nhóm tấn công [42]; UNSW-NB15 - toàn bộ tập huấn luyện [45]; UGR16 - phần tuần thứ tư [50]; ADFA-NetflowIDS - toàn bộ tập dữ liệu [51]; CICIDS17 - một phần nhật ký Thứ ba, Thứ tư và chiều Thứ sáu để bao gồm bốn nhóm tấn công [53]; và CIDDS-001 - các tập “9” và “20” [44]. Ngoài ra, NSL-KDD được đưa vào với toàn bộ phần huấn luyện và kiểm thử [43] vì vẫn thường được dùng làm chuẩn so sánh, dù đã hơn mười năm tuổi.

### Các tập dữ liệu có tấn công chưa biết

Để phân tích khả năng phát hiện tấn công chưa biết, với mỗi tập dữ liệu  $dn$ , nhóm tác giả tạo các biến thể  $dn\_att$ , trong đó  $att$  là một nhóm tấn công thuộc  $dn$ . Mỗi biến thể được chia 50% huấn luyện và 50% kiểm thử; phần huấn luyện không chứa điểm dữ liệu của tấn công  $att$ . Vì vậy, trong giai đoạn huấn luyện,  $att$  là loại chưa biết và khi kiểm thử có vai trò tương đương tấn công zero-day.

Ví dụ, ISCX12\_BruteForce là biến thể trong đó: (i) phần huấn luyện gồm dữ liệu bình thường và tất cả các tấn công trừ BruteForce, tức DoS, DDoS và Infiltration; (ii) phần kiểm thử gồm dữ liệu bình thường cùng cả DoS, DDoS, Infiltration và BruteForce. Tổng cộng có 33 biến thể, mỗi biến thể tương ứng một nhóm tấn công: 5 biến thể của ADFA-Netflow, 4 của CIDDS-001, 4 của ISCX12, 4 của NSL-KDD, 5 của UGR16, 4 của CICIDS17 và 7 của UNSW-NB15. Như vậy, thí nghiệm dùng 7 tập dữ liệu đầy đủ và 33 biến thể, tổng cộng 40 tập.

### 5.3. Bộ phân loại không giám sát ở tầng cơ sở

Nhóm tác giả chọn một tập bộ phân loại nhị phân không giám sát để phát hiện xâm nhập trên các tập dữ liệu đã chọn. Để bảo đảm tính đa dạng của bộ học cơ sở, các thuật toán được chọn từ nhiều họ: dựa trên góc, phân cụm, mạng nơ-ron, mật độ, láng giềng, thống kê và phân loại. Các thuật toán quá nặng, chẳng hạn ABOD có độ phức tạp bậc ba [46], bị loại vì bản thân học meta đã cần nhiều tài nguyên tính toán và bộ nhớ.

Bộ 14 thuật toán gồm:

- Một thuật toán đại diện cho mỗi họ: One-Class SVM cho họ phân loại [62], K-Means cho phân cụm [73], biến thể không giám sát của kNN cho họ láng giềng [71], HBOS cho thống kê [60], SOM cho mạng nơ-ron [64], FastABOD cho họ dựa trên góc [46] và LOF cho mật độ [70].
- Các thuật toán phổ biến bổ sung để mở rộng phạm vi: DBSCAN [69], COF [67], LDcoF [66], Isolation Forest [72], G-Means [65], ODIN [61] và Sparse Density Observers - SDO [63].

**Bảng 2. Thuật toán không giám sát dùng làm bộ học cơ sở để tạo đặc trưng meta dựa trên mô hình. Tham số và độ phức tạp tương ứng hiện thực trong công cụ RELOAD [52].**

| Thuật toán | Họ                 | Tham số                            | Độ phức tạp huấn luyện | Độ phức tạp kiểm thử |
|------------|--------------------|------------------------------------|------------------------|----------------------|
| COF        | Mật độ, láng giềng | $k$ : số láng giềng                | $O(kn \log n)$         | $O(k \log n)$        |
| DBSCAN     | Phân cụm           | $\minPts$ , $\epsilon$             | $O(n \log n)$          | $O(1)$               |
| FastABOD   | Góc, láng giềng    | $k$ : số láng giềng                | $O(n^2 + nk^2)$        | $O(k^2 \log n)$      |
| G-Means    | Phân cụm           | tự động tìm số cụm $c$             | $O(cn)$                | $O(c)$               |
| HBOS       | Thống kê           | $b$ : số bin                       | $O(n)$                 | $O(1)$               |
| iForest    | Phân loại          | $t$ : số cây; $s$ : số mẫu mỗi cây | $O(ts \log s)$         | $O(t \log s)$        |
| K-Means    | Phân cụm           | $c$ : số cụm                       | $O(cn)$                | $O(c)$               |
| kNN        | Láng giềng         | $k$ : số láng giềng                | $O(n \log n)$          | $O(k \log n)$        |
| LDcoF      | Mật độ, phân cụm   | $c$ : số cụm                       | $O(c^2 + cn)$          | $O(c^2)$             |
| LOF        | Mật độ, láng giềng | $k$ : số láng giềng                | $O(kn + n \log n)$     | $O(k \log n)$        |
| ODIN       | Láng giềng         | $k$ : số láng giềng                | $O(n \log n)$          | $O(k \log n)$        |
| SDO        | Mật độ             | $obs$ : số observer; $q$           | $O(obs \cdot n)$       | $O(obs)$             |
| SOM        | Mạng nơ-ron        | $e$ : số epoch; $x$ : số nơ-ron    | $O(nex)$               | $O(x)$               |

| Thuật toán    | Họ        | Tham số                    | Độ phức tạp huấn luyện | Độ phức tạp kiểm thử |
|---------------|-----------|----------------------------|------------------------|----------------------|
| One-Class SVM | Phân loại | kernel; nu: tỷ lệ lỗi biên | $O(n^2)$               | $O(1)$               |

Các thuật toán có độ phức tạp huấn luyện nhiều nhất là bậc hai và thời gian kiểm thử hằng số, ngoại trừ nhóm dựa trên láng giềng phải tìm kNN và cần ít nhất thời gian logarit nếu dùng cây K-D. Sau khi xem xét các framework công khai cho phép chạy thuật toán không giám sát trên tập dữ liệu, nhóm tác giả chọn RELOAD [52], một công cụ Java mã nguồn mở bao bọc các thuật toán từ ELKI và WEKA, đồng thời cung cấp thêm một số hiện thực. RELOAD còn hỗ trợ grid search để tìm tham số tối ưu.

Ngoại trừ G-Means không có tham số, các cấu hình được thử như sau:

- Với thuật toán dựa trên kNN, k; với iForest, số mẫu s và số cây t; với HBOS, số histogram/bin; và với SDO, số observer obs được chọn trong {1, 2, 3, 5, 10, 20, 50, 100, 200, 500}.
- One-Class SVM dùng kernel {tuyến tính, bậc hai, bậc ba, RBF} và nu trong {0,01; 0,02; 0,05; 0,1; 0,2}. Tham số nu ảnh hưởng số vector hỗ trợ được tạo.
- Ngoài obs, SDO dùng ngưỡng q trong {0,05; 0,1; 0,2; 0,5} để xác định các observer “gần nhất”.
- DBSCAN dùng minPts trong {1, 2, 3, 5, 10} và eps trong {100, 200, 500, 1000}, với eps là bán kính cụm quanh mỗi điểm dữ liệu.

Với mỗi thuật toán và tập dữ liệu, grid search tìm cấu hình cho MCC tốt nhất trên một phần dữ liệu nhỏ dùng làm validation và không chồng lấn với tập kiểm thử. Toàn bộ quá trình được RELOAD tự động quản lý.

## 5.4. Chỉ số đánh giá

Hiệu quả của bộ phân loại nhị phân thường được đánh giá bằng số phân loại đúng - dương tính thật (TP), âm tính thật (TN) - và phân loại sai - âm tính giả (FN), dương tính giả (FP) - tạo thành ma trận nhầm lẫn. Theo [49], các chỉ số thông dụng tổng hợp các phần tử này để tạo Precision, Recall hay Coverage, tỷ lệ dương tính giả (FPR), Accuracy (ACC), F-score beta, F1, diện tích dưới đường ROC (AUC) và hệ số tương quan Matthews (MCC).

Mỗi chỉ số có ưu, nhược điểm riêng và phải được xem xét theo miền bài toán. IDS trước hết cần giảm FN, tức các cuộc tấn công không được phát hiện. Tuy vậy, một IDS quá “đa nghi” có thể giảm mạnh FN bằng cách tăng FP; dù phát hiện tốt, nó tạo quá nhiều cảnh báo giả. Vì vậy, bài báo chủ yếu dùng:

- **Tỷ lệ phân loại sai (Misc):** phần trăm điểm dữ liệu bị phân loại sai trên toàn tập,  $Misc = 1 - Accuracy$ .
- **MCC [48]:** tổng hợp tất cả ô của ma trận nhầm lẫn và phù hợp cả với dữ liệu mất cân bằng [47], chẳng hạn tập chủ yếu là bình thường và chỉ có ít tấn công.
- **Recall [49]:** chỉ số định hướng FN, biểu thị tỷ lệ tấn công được phát hiện trên tổng số tấn công. Khi thích hợp, bài báo còn tính **Recall-Unk**, chỉ xét khả năng phát hiện các cuộc tấn công không có trong tập huấn luyện.

## 5.5. Các tập đặc trưng meta

Gọi:

$UA = \{COF, DBSCAN, FastABOD, G-Means, HBOS, iForest, K-Means, kNN, LDCOF, LOF, ODIN, SDO, SOM, One-Class SVM\}$

là tập 14 thuật toán không giám sát ở Mục 5.3; và rep là mảng 14 phần tử chứa MCC trong  $[-1, 1]$  của từng thuật toán trên một tập dữ liệu. Để khảo sát ảnh hưởng của các tập đặc trưng tới lỗi của bộ phân loại tầng meta và toàn bộ Stacker, nghiên cứu định nghĩa:

$$DataF = \{f \in Dataset \mid f \text{ chứa giá trị thứ tự}\}$$

$$MetaF(A) = \{ua.num, ua.bin \mid ua \in A\} \cup Count(A) \cup WCount(A, rep(A))$$

$$FullF(A) = DataF \cup MetaF(A)$$

Tóm lại, DataF gồm tất cả đặc trưng thứ tự của tập dữ liệu đích. MetaF(A) gồm ua.num, ua.bin của mọi thuật toán, cộng Count(A) và WCount(A, rep(A)). FullF(A) hợp nhất đặc trưng gốc và đặc trưng meta.

## 5.6. Bộ phân loại có giám sát ở tầng meta

Bộ phân loại có giám sát được dùng làm bộ học tầng meta. Nghiên cứu chọn kNN [39], SVM [33], cây quyết định, Random Forest [31], AdaBoost [32], Naive Bayes và phân tích biệt thức bậc hai - QDA [40], đều sử dụng hiện thực trong thư viện Scikit-learn. Cấu hình tốt nhất được chọn bằng grid search, tương tự RELOAD:

- kNN với k trong {1, 3, 10, 30, 100} và khoảng cách Euclid.
- Ba SVM riêng biệt dùng kernel tuyến tính, RBF hoặc đa thức bậc hai.
- Cây quyết định có giới hạn độ sâu {5, 20, không giới hạn}.
- AdaBoostM2 với {10, 30, 100, 200} cây.
- Random Forest với {10, 30, 100} cây, mỗi cấu hình dùng cây không giới hạn độ sâu hoặc giới hạn độ sâu bằng 10.
- Naive Bayes và QDA dùng tham số mặc định của Scikit-learn.

Ngoài ra, nghiên cứu xem xét mạng nơ-ron có **entity embedding**, giúp phù hợp dữ liệu dạng bảng khi dữ liệu thưa hoặc phân bố thống kê chưa biết. Entity embedding ánh xạ các giá trị tương tự gần nhau trong không gian embedding, nhờ đó bộc lộ tính chất nội tại của biến phân loại rời rạc [58]. Mã Python được xây dựng từ [59] và công bố tại [93].

## 5.7. Thiết lập và thực hiện thí nghiệm

### Chọn đặc trưng cho bộ học cơ sở

Information Gain [91] được dùng để tự động xếp hạng và chọn các đặc trưng liên quan nhất của từng tập dữ liệu, loại bỏ đặc trưng nhiễu.

### Chọn đặc trưng cho bộ học tầng meta

Các thuật toán có giám sát được phép tự động chọn và suy ra đặc trưng, như representation learning trong mô hình sâu [58], bằng cách cung cấp toàn bộ tập đặc trưng meta.

### Chia tập huấn luyện - kiểm thử

Đặc trưng dựa trên mô hình không giám sát được tạo cho tập dữ liệu đầy đủ và các biến thể bằng phép chia 30%-70% huấn luyện - kiểm thử với lấy mẫu 10-fold; đây là phân huấn luyện lớn nhất không làm công cụ RELOAD viết bằng Java phát sinh lỗi bộ nhớ. Với các thuật toán có giám sát tầng meta, dữ liệu được chia 50%-50%.

### Máy thực hiện thí nghiệm

Toàn bộ chiến dịch thí nghiệm được chạy trên máy chủ Dell Precision 5820 Tower, CPU 12 nhân Intel Core i9-9920X, GPU Nvidia Quadro RTX5000 và 2,5 TB dung lượng lưu trữ người dùng.

### Thời gian thực hiện

Tổng thời gian chạy liên tục 24 giờ mỗi ngày xấp xỉ 50 ngày. Tất cả tệp và điểm số chỉ số được dùng để tổng hợp kết quả được công bố tại [94].

## 6. SO SÁNH STACKER VỚI BỘ PHÂN LOẠI CÓ GIÁM SÁT VÀ KHÔNG GIÁM SÁT

Mục này phân tích kết quả thực nghiệm để làm rõ mức cải thiện khi dùng Stacker thay cho chỉ một thuật toán ML có giám sát hoặc không giám sát trong phát hiện xâm nhập.

### 6.1. Thông tin do đặc trưng meta không giám sát mang lại

Trước hết, nghiên cứu xem xét liệu các đặc trưng meta không giám sát có cung cấp thông tin bổ sung giúp giảm phân loại sai hay không. Bảng 3 trình bày Information Gain cao nhất - cột “1”, tức giá trị cao nhất của một đặc trưng - và Information Gain trung bình của 3, 5 và 10 đặc trưng tốt nhất. Các giá trị được tính cho DataF, MetaF(UA) và FullF(UA) trên từng tập dữ liệu.

**Bảng 3. Information Gain của 1, 3, 5 và 10 đặc trưng tốt nhất trong từng tập đặc trưng**

| Tập dữ liệu | Tập đặc trưng | Số đặc trưng | 1     | 3     | 5     | 10    |
|-------------|---------------|--------------|-------|-------|-------|-------|
| ADFA Net    | DataF         | 3            | 0,859 | 0,381 | 0,381 | 0,381 |
| ADFA Net    | MetaF(UA)     | 30           | 0,865 | 0,865 | 0,864 | 0,860 |
| ADFA Net    | FullF(UA)     | 33           | 0,865 | 0,865 | 0,864 | 0,861 |
| CICIDS17    | DataF         | 75           | 0,491 | 0,478 | 0,447 | 0,407 |
| CICIDS17    | MetaF(UA)     | 30           | 0,445 | 0,440 | 0,438 | 0,434 |
| CICIDS17    | FullF(UA)     | 105          | 0,491 | 0,478 | 0,463 | 0,449 |
| CIDDS-001   | DataF         | 7            | 0,886 | 0,708 | 0,527 | 0,387 |
| CIDDS-001   | MetaF(UA)     | 30           | 0,855 | 0,818 | 0,788 | 0,737 |
| CIDDS-001   | FullF(UA)     | 37           | 0,886 | 0,855 | 0,828 | 0,773 |
| ISCX12      | DataF         | 6            | 0,339 | 0,317 | 0,272 | 0,248 |
| ISCX12      | MetaF(UA)     | 30           | 0,319 | 0,303 | 0,248 | 0,192 |
| ISCX12      | FullF(UA)     | 36           | 0,339 | 0,331 | 0,317 | 0,260 |
| NSL-KDD     | DataF         | 37           | 0,746 | 0,657 | 0,588 | 0,500 |
| NSL-KDD     | MetaF(UA)     | 30           | 0,700 | 0,671 | 0,652 | 0,635 |
| NSL-KDD     | FullF(UA)     | 67           | 0,746 | 0,703 | 0,677 | 0,648 |
| UGR16       | DataF         | 7            | 0,442 | 0,285 | 0,201 | 0,147 |
| UGR16       | MetaF(UA)     | 30           | 0,402 | 0,399 | 0,394 | 0,382 |
| UGR16       | FullF(UA)     | 37           | 0,442 | 0,413 | 0,406 | 0,390 |
| UNSW-NB15   | DataF         | 38           | 0,592 | 0,520 | 0,428 | 0,319 |
| UNSW-NB15   | MetaF(UA)     | 30           | 0,651 | 0,613 | 0,595 | 0,553 |
| UNSW-NB15   | FullF(UA)     | 68           | 0,651 | 0,615 | 0,602 | 0,563 |

Xét UNSW-NB15 ở cuối bảng, Information Gain trung bình của FullF(UA) cao hơn DataF, vốn không chứa đặc trưng meta dựa trên mô hình. Xu hướng này cũng xuất hiện ở đặc trưng tốt nhất, cột “1”: giá trị của FullF(UA) cao hơn DataF và bằng MetaF(UA), nghĩa là đặc trưng nhiều thông tin nhất là một đặc trưng meta, cụ thể là WCount(UA). Có thể kỳ vọng bộ phân loại tăng meta sẽ dựa vào đặc trưng này khi xây dựng mô hình và nhờ đó cải thiện phân loại.

Hành vi tương tự xuất hiện với ADFA Net ở đầu bảng. Tập này chỉ có ba đặc trưng thứ tự, nên: (i) trung bình của 3, 5 và 10 đặc trưng tốt nhất trong DataF là như nhau; và (ii) bổ sung đặc trưng meta có nhiều khả năng mang lại thông tin hữu ích vì số đặc trưng tăng mạnh từ 3 lên 33.

Nhìn chung, lượng thông tin trong FullF(UA) cao hơn đáng kể DataF trên tất cả các tập khi xét 5 và 10 đặc trưng, và thường cải thiện cả đặc trưng tốt nhất hoặc ba đặc trưng tốt nhất. Phân tích định lượng này cho thấy các đặc trưng meta mang thông tin có giá trị và có tiềm năng tăng chất lượng phân loại.

## 6.2. Thảo luận và so sánh khả năng phát hiện tấn công

Bảng 4 so sánh tỷ lệ phân loại sai, MCC, Recall và Recall-Unk của Stacker với các thuật toán có giám sát và không giám sát.

Với mỗi chỉ số, bảng gồm:

1. **Sup - DataF:** kết quả tốt nhất của các thuật toán có giám sát ở Mục 5.6 khi dùng đặc trưng DataF.
2. **Stacker - MetaF(UA):** kết quả Stacker khi chỉ dùng MetaF(UA).
3. **Stacker - FullF(UA):** kết quả Stacker khi dùng FullF(UA).
4. **Unsup - DataF:** kết quả tốt nhất của các thuật toán không giám sát ở Mục 5.3 khi dùng DataF.

Các giá trị trong bảng là kết quả của bộ phân loại đạt MCC cao nhất trên từng tập dữ liệu hoặc biến thể. Recall-Unk không được tính riêng cho Unsup - DataF, vì thuật toán không giám sát không phân biệt tấn công đã biết với chưa biết, nên việc tách Recall khỏi Recall-Unk không có ý nghĩa.

Table 4: Misclassifications, MCC, Recall, Recall\_Unk (recall on unknowns-only) achieved by algorithms on each of the 7 dataset and their variants. For each metric, we report scores obtained by using Supervised algorithms with DataF features, Stacker with MetaF(UA) and FullF(UA) features, and Unsupervised Algorithms using DataF features.

| Dataset    | Unknown Attacks (Variant) | % Unkn | Misclassifications (%) |            |            |       | MCC   |            |            |       | Recall |            |            |       | Recall-Unk |            |            |
|------------|---------------------------|--------|------------------------|------------|------------|-------|-------|------------|------------|-------|--------|------------|------------|-------|------------|------------|------------|
|            |                           |        | Sup                    | Stacker    |            | Unsup | Sup   | Stacker    |            | Unsup | Sup    | Stacker    |            | Unsup | Sup        | Stacker    |            |
|            |                           |        | DataF                  | MetaF (UA) | FullF (UA) | DataF | DataF | MetaF (UA) | FullF (UA) | DataF | DataF  | MetaF (UA) | FullF (UA) | DataF | DataF      | MetaF (UA) | FullF (UA) |
| ADFANet    | -                         | 0.0    | 1.33                   | 0.32       | 0.44       | 2.41  | .969  | .992       | .990       | .945  | .996   | .998       | .999       | .984  | -          | -          | -          |
|            | Mailbomb                  | 0.8    | 1.96                   | 0.32       | 0.44       | 1.33  | .956  | .992       | .990       | .970  | .996   | .998       | .999       | .997  | 1.000      | 1.000      | 1.000      |
|            | Other                     | 2.7    | 1.96                   | 1.17       | 1.17       | 1.33  | .956  | .973       | .973       | .970  | .980   | .980       | .972       | .997  | .750       | .776       | .776       |
|            | PortswEEP                 | 3.8    | 1.96                   | 0.32       | 0.44       | 0.70  | .956  | .992       | .990       | .984  | .996   | .998       | .998       | 1.000 | 1.000      | 1.000      | 1.000      |
|            | 1b                        | 10.3   | 0.47                   | 0.95       | 0.60       | 0.67  | .989  | .978       | .986       | .984  | .991   | .986       | .964       | 1.000 | .980       | .974       | .960       |
|            | Neptune                   | 13.4   | 1.96                   | 0.32       | 0.44       | 0.70  | .956  | .992       | .990       | .984  | .996   | .998       | .999       | 1.000 | 1.000      | 1.000      | 1.000      |
| CICIDS17   | -                         | 0.0    | 0.03                   | 0.10       | 0.03       | 1.85  | .999  | .959       | .999       | .915  | .959   | .998       | .998       | .861  | -          | -          | -          |
|            | Patator                   | 1.0    | 0.76                   | 1.60       | 1.02       | 1.90  | .966  | .927       | .954       | .912  | .878   | .921       | .941       | .857  | .295       | .295       | .295       |
|            | PortScan                  | 11.2   | 11.14                  | 6.81       | 11.12      | 3.66  | .346  | .722       | .349       | .923  | .862   | .138       | .138       | .958  | .007       | .992       | .806       |
|            | DoS                       | 17.7   | 0.75                   | 1.00       | 0.75       | 2.41  | .966  | .955       | .966       | .888  | .935   | .942       | .942       | .858  | .000       | .000       | .000       |
| CICIDS-001 | -                         | 0.0    | 0.97                   | 1.27       | 0.19       | 12.15 | .980  | .974       | .996       | .746  | .998   | .998       | .997       | .753  | -          | -          | -          |
|            | BruteForce                | 0.1    | 0.82                   | 1.27       | 1.27       | 12.31 | .983  | .974       | .974       | .746  | .997   | .997       | .997       | .876  | .000       | .126       | .077       |
|            | PingScan                  | 0.2    | 0.84                   | 1.28       | 0.21       | 13.74 | .982  | .974       | .996       | .713  | .997   | .997       | .997       | .837  | .978       | .988       | .963       |
|            | PortScan                  | 9.4    | 9.15                   | 9.41       | 3.35       | 11.89 | .815  | .810       | .931       | .758  | .762   | .919       | .763       | .718  | .060       | .661       | .673       |
|            | DoS                       | 29.5   | 29.31                  | 30.73      | 25.00      | 6.55  | .405  | .356       | .489       | .726  | .245   | .392       | .535       | .821  | .025       | .318       | .195       |
| ISCX12     | -                         | 0.0    | 3.55                   | 3.21       | 1.20       | 5.94  | .796  | .761       | .916       | .489  | .670   | .903       | .899       | .290  | -          | -          | -          |
|            | DoS                       | 0.6    | 3.03                   | 3.22       | 1.67       | 6.64  | .803  | .758       | .882       | .464  | .641   | .830       | .861       | .386  | .004       | .392       | .214       |
|            | BruteForce                | 0.9    | 3.85                   | 3.12       | 1.45       | 6.54  | .766  | .768       | .897       | .484  | .686   | .838       | .702       | .421  | .001       | .222       | .270       |
|            | DDoS                      | 3.1    | 3.86                   | 5.76       | 3.77       | 6.00  | .702  | .524       | .711       | .480  | .378   | .556       | .538       | .263  | .000       | .231       | .322       |
|            | Infiltration              | 3.4    | 6.43                   | 3.21       | 3.27       | 5.52  | .536  | .761       | .759       | .544  | .673   | .685       | .743       | .385  | .530       | .538       | .539       |
| NSL-KDD    | -                         | 0.0    | 0.55                   | 1.10       | 0.61       | 9.07  | .989  | .978       | .988       | .817  | .993   | .997       | .996       | .877  | -          | -          | -          |
|            | u2r                       | 0.1    | 0.58                   | 1.13       | 0.63       | 9.89  | .988  | .977       | .987       | .801  | .993   | .996       | .997       | .876  | .267       | .400       | .550       |
|            | r2l                       | 2.8    | 2.57                   | 2.87       | 2.46       | 9.36  | .949  | .942       | .951       | .812  | .954   | .959       | .963       | .868  | .056       | .129       | .155       |
|            | Probe                     | 9.5    | 4.44                   | 4.49       | 4.88       | 10.94 | .911  | .910       | .902       | .780  | .923   | .923       | .919       | .853  | .282       | .604       | .651       |
|            | DoS                       | 12.4   | 5.03                   | 5.34       | 4.95       | 8.86  | .901  | .893       | .903       | .824  | .836   | .889       | .907       | .850  | .866       | .896       | .865       |
| UGR16      | -                         | 0.0    | 0.44                   | 2.19       | 0.33       | 3.98  | .980  | .907       | .985       | .759  | .953   | .975       | .977       | .808  | -          | -          | -          |
|            | BlackList                 | 0.4    | 0.71                   | 2.47       | 0.73       | 7.01  | .969  | .893       | .968       | .708  | .925   | .944       | .931       | .797  | .000       | .040       | .000       |
|            | NerisBotnet               | 0.4    | 0.65                   | 2.88       | 0.66       | 6.21  | .971  | .872       | .971       | .734  | .880   | .886       | .952       | .798  | .015       | .000       | .000       |
|            | Anom-Spam                 | 0.7    | 1.08                   | 2.54       | 1.14       | 5.15  | .952  | .889       | .949       | .771  | .915   | .912       | .914       | .796  | .000       | .000       | .000       |
|            | DoS                       | 2.3    | 1.64                   | 3.24       | 0.25       | 6.38  | .926  | .856       | .989       | .733  | .867   | .982       | .974       | .815  | .514       | 1.000      | 1.000      |
|            | Scan44                    | 9.2    | 6.01                   | 4.93       | 4.42       | 6.74  | .709  | .765       | .792       | .720  | .641   | .715       | .880       | .809  | .380       | .788       | .702       |
| UNSW-NB15  | -                         | 0.0    | 6.76                   | 5.95       | 2.81       | 17.31 | .862  | .879       | .941       | .653  | .971   | .982       | .981       | .870  | -          | -          | -          |
|            | Worms                     | 0.1    | 6.73                   | 6.01       | 3.04       | 17.57 | .862  | .878       | .937       | .626  | .971   | .984       | .981       | .763  | .985       | 1.000      | 1.000      |
|            | Shellcode                 | 0.7    | 6.81                   | 5.89       | 2.82       | 17.77 | .862  | .880       | .941       | .622  | .970   | .981       | .980       | .757  | .984       | .956       | .984       |
|            | Backdoor                  | 1.1    | 6.15                   | 5.84       | 2.95       | 17.43 | .873  | .881       | .939       | .630  | .970   | .984       | .981       | .766  | 1.000      | .993       | .999       |
|            | Analysis                  | 1.2    | 6.10                   | 5.93       | 2.90       | 17.76 | .874  | .879       | .939       | .622  | .968   | .981       | .973       | .758  | .833       | .985       | .907       |
|            | Reconnaiss.               | 6.3    | 6.58                   | 6.14       | 2.93       | 17.77 | .866  | .874       | .939       | .622  | .964   | .982       | .980       | .757  | .977       | .968       | .996       |
|            | DoS                       | 7.4    | 7.46                   | 6.02       | 3.67       | 17.71 | .848  | .877       | .924       | .623  | .967   | .982       | .980       | .757  | .967       | .988       | .995       |
|            | Exploits                  | 10.3   | 5.76                   | 5.55       | 2.81       | 17.42 | .879  | .886       | .941       | .630  | .968   | .978       | .976       | .766  | .968       | .992       | .990       |
|            | Fuzzers                   | 11.0   | 10.37                  | 7.57       | 9.08       | 16.27 | .784  | .839       | .811       | .659  | .855   | .782       | .781       | .811  | .198       | .561       | .272       |
|            | Averages                  |        | 4.26                   | 4.09       | 2.80       | 8.62  | .870  | .870       | .911       | .746  | .862   | .913       | .900       | .785  | .482       | .631       | .611       |

Bảng 4

**Bảng 4.** Tỷ lệ phân loại sai, MCC, Recall và Recall-Unk trên 7 tập dữ liệu cùng các biến thể. Chú giải tiêu đề: Unknown Attacks (Variant) = Tấn công chưa biết/biến thể; Unkn % = tỷ lệ tấn công chưa biết; Sup = thuật toán có giám sát; Unsup = thuật toán không giám sát; MetaF = tập đặc trưng meta; FullF = đặc trưng gốc cộng đặc trưng meta. Dòng “Averages” là trung bình trên toàn bộ 40 tập/biến thể.

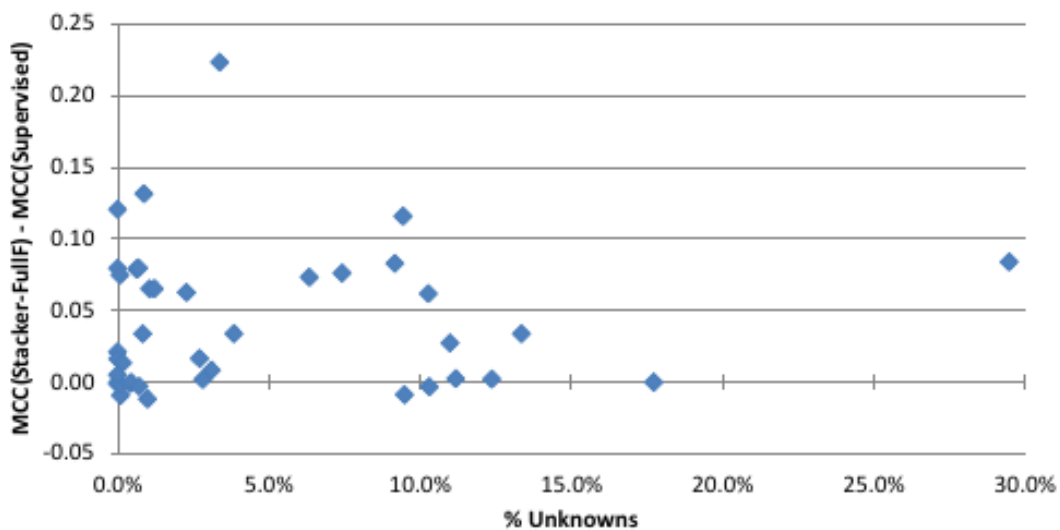
Tỷ lệ Misc thấp cho thấy phần lớn điểm dữ liệu được phân loại đúng. Toàn bộ tập và các biến thể của ADFANet, ISCX12 và UGR16 đều có dưới 5% phân loại sai. Ngược lại, CIDDS\_DoS có tỷ lệ rất cao, khoảng 30% với thuật toán có giám sát và Stacker, dù dùng MetaF(UA) hay FullF(UA). CICIDS17\_PortScan và UNSW\_Fuzzers có tỷ lệ quanh 10%, nghĩa là trung bình cứ mười điểm dữ liệu có một điểm bị phân loại sai.

Lỗi của thuật toán không giám sát không dao động nhiều giữa các biến thể của cùng một tập dữ liệu, nhưng nhìn chung cao hơn thuật toán có giám sát và Stacker. Nguyên nhân là quá trình huấn luyện không có nhãn, nên việc thêm điểm dữ liệu của một tấn công cụ thể ít ảnh hưởng hơn đến mô hình hành vi bình thường so với huấn luyện có giám sát.

Xu hướng rõ nhất ở dòng trung bình: thuật toán có giám sát có tỷ lệ phân loại sai trung bình 4,26%, trong khi thuật toán không giám sát là 8,62%. Stacker-FullF(UA) chỉ còn 2,80%, giảm khoảng 33% so với thuật toán có giám sát. MCC cũng xác nhận xu hướng: thuật toán không giám sát có trung bình thấp nhất, 0,746; còn Stacker-FullF(UA) đạt cao nhất, 0,911. MCC càng cao càng tốt.

Hình 4 biểu diễn chênh lệch MCC giữa Stacker-FullF(UA) và Sup-DataF theo tỷ lệ tấn công chưa biết trong tập kiểm thử. Việc dùng Stacker nhìn chung có lợi vì gần như mọi điểm đều nằm trên 0. Có trường hợp mức tăng rất lớn: với ISCX\_Infiltration, MCC tăng từ 0,536 của thuật toán có giám sát lên 0,759 với Stacker. Có trường hợp chênh lệch nhỏ nhưng rất ý nghĩa: trên tập CIDDS đầy đủ, Sup-DataF đạt MCC 0,98 với khoảng 1% lỗi, còn Stacker-FullF(UA) đạt MCC 0,996 với chỉ 0,2% lỗi. Tổng cộng, Stacker-FullF(UA) có MCC cao nhất trên 22 trong 40 tập/biến thể và vượt Sup-DataF trên 30 tập/biến thể, tức 75% trường hợp.

**Figure 4: Difference of MCC achieved by Stacker-FullF(UA) and Sup-DataF plotted against % of unknowns in the test set.**



**Hình 4**

**Hình 4.** Chênh lệch  $MCC(\text{Stacker-FullF}) - MCC(\text{Supervised})$  theo tỷ lệ tấn công chưa biết trong tập kiểm thử. Phần lớn điểm nằm trên 0, cho thấy Stacker cải thiện MCC.

### 6.3. Stacker và tấn công zero-day

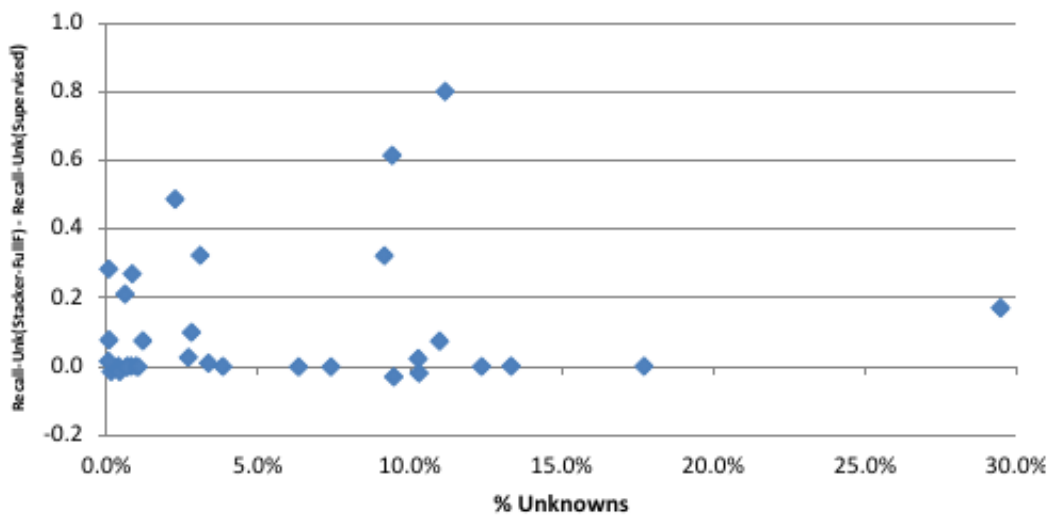
Tiếp theo, nghiên cứu tập trung vào các cuộc tấn công bị bỏ sót, tức FN, và chỉ số Recall. Recall trong Bảng 4 có xu hướng tương tự MCC: Unsup-DataF thấp hơn Sup-DataF, trong khi Stacker nhìn chung cải thiện Recall và đạt trung bình tốt nhất. Với phát hiện zero-day, Recall-Unk cho thấy:

- Recall và Recall-Unk của bộ phân loại có giám sát khác nhau rõ rệt, xác nhận chúng yếu trong phát hiện tấn công zero-day.

- Recall của thuật toán không giám sát thấp hơn đáng kể thuật toán có giám sát và Stacker, nhưng cao hơn rõ rệt Recall-Unk của thuật toán có giám sát. Điều này xác nhận thuật toán không giám sát tạo nhiều phân loại sai, nhưng tạo ít FN hơn khi gặp zero-day.
- Tương tự Misc và MCC, Stacker - đặc biệt với FullF(UA) - cải thiện cả Recall và Recall-Unk so với Sup-DataF.

Hình 5 biểu diễn chênh lệch Recall-Unk theo tỷ lệ tấn công chưa biết. Chênh lệch gần như luôn dương, củng cố lý do sử dụng Stacker. Xu hướng tương tự Hình 4 nhưng độ chênh tuyệt đối thường lớn hơn, có thể tới 0,8.

Figure 5: Difference of Recall-Unk achieved by Stacker-FullF(UA) against Sup-DataF plotted against % of unknowns in the test set.



Hình 5

**Hình 5.** Chênh lệch Recall-Unk(Stacker-FullF) - Recall-Unk(Supervised) theo tỷ lệ tấn công chưa biết. Giá trị gần như luôn dương.

## 6.4. Bộ phân loại có giám sát làm bộ học tầng meta

Kết quả cho thấy Stacker dùng FullF(UA) tối đa hóa hiệu năng phát hiện xâm nhập. Tuy nhiên, cần xác định bộ phân loại có giám sát phù hợp nhất cho tầng meta. Bảng 5 trình bày MCC trung bình, độ lệch chuẩn MCC và số lần mỗi cấu hình đạt MCC tốt nhất trên 40 tập/biến thể.

**Bảng 5.** Các bộ phân loại có giám sát được xem xét cho tầng meta

| Thuật toán tầng meta | Tham số               | MCC trung bình | Độ lệch chuẩn MCC | Số lần tốt nhất |
|----------------------|-----------------------|----------------|-------------------|-----------------|
| AdaBoost             | 10 cây                | 0,883          | 0,081             | 1               |
| AdaBoost             | 30 cây                | 0,886          | 0,081             | 1               |
| AdaBoost             | 100 cây               | 0,795          | 0,276             | 0               |
| AdaBoost             | 200 cây               | 0,764          | 0,325             | 1               |
| kNN                  | 1-NN                  | 0,906          | 0,077             | 1               |
| kNN                  | 3-NN                  | 0,902          | 0,081             | 0               |
| kNN                  | 10-NN                 | 0,899          | 0,079             | 5               |
| kNN                  | 30-NN                 | 0,903          | 0,084             | 0               |
| kNN                  | 100-NN                | 0,897          | 0,093             | 3               |
| Cây quyết định       | Không giới hạn độ sâu | 0,750          | 0,211             | 0               |
| Cây quyết định       | Độ sâu 20             | 0,751          | 0,207             | 2               |
| Cây quyết định       | Độ sâu 5              | 0,801          | 0,171             | 0               |

| Thuật toán tầng meta | Tham số                                        | MCC trung bình | Độ lệch chuẩn MCC | Số lần tốt nhất |
|----------------------|------------------------------------------------|----------------|-------------------|-----------------|
| MLP                  | Mặc định                                       | 0,815          | 0,189             | 0               |
| Mạng có embedding    | 3 tầng dense, batch normalization, dropout 0,5 | 0,899          | 0,037             | 10              |
| Naive Bayes          | Mặc định                                       | 0,674          | 0,312             | 0               |
| QDA                  | Mặc định                                       | 0,538          | 0,472             | 0               |
| SVM                  | Kernel tuyến tính                              | 0,844          | 0,092             | 0               |
| SVM                  | Kernel bậc hai                                 | 0,866          | 0,097             | 0               |
| SVM                  | Kernel RBF                                     | 0,853          | 0,095             | 0               |
| Random Forest        | 10 cây, không giới hạn độ sâu                  | 0,904          | 0,062             | 0               |
| Random Forest        | 10 cây, độ sâu 10                              | 0,912          | 0,053             | 2               |
| Random Forest        | 30 cây, không giới hạn độ sâu                  | 0,915          | 0,056             | 1               |
| Random Forest        | 30 cây, độ sâu 10                              | 0,915          | 0,056             | 1               |
| Random Forest        | 100 cây, không giới hạn độ sâu                 | 0,915          | 0,061             | 12              |
| Random Forest        | 100 cây, độ sâu 10                             | 0,912          | 0,066             | 0               |

Mạng nơ-ron có embedding và Random Forest 100 cây không giới hạn độ sâu là những lựa chọn tốt nhất. Random Forest vượt các thuật toán khác trên 12 trong 40 tập/biến thể và có MCC trung bình cao nhất; nếu xét toàn bộ các cấu hình Random Forest, chúng đạt tốt nhất trong 16 trường hợp. Nhiều cấu hình kNN có MCC trung bình trên 0,90 nhưng độ biến thiên cao hơn Random Forest. Mạng có embedding có độ lệch chuẩn thấp nhất, nên là lựa chọn cân bằng. Naive Bayes và QDA biến thiên rất cao, không đủ tin cậy; AdaBoost và MLP tạo nhiều phân loại sai. Vì vậy, bài báo khuyến nghị **Random Forest 100 cây** hoặc **mạng nơ-ron có embedding** làm bộ học tầng meta.

## 7. MỘT STACKER CHO PHÁT HIỆN XÂM NHẬP

Dựa trên kết quả trên, bài báo hiện thực một Stacker sử dụng tập đặc trưng FullF(UA) như Hình 6.

### 7.1. Kiến trúc tổng thể

Quá trình phát hiện bắt đầu từ hệ thống đích, bước 1 ở góc trên trái Hình 6. Các probe giám sát dữ liệu mạng và ghi log để tạo tập đặc trưng DataF, bước 2. Những đặc trưng này cùng MetaF(UA) tạo FullF(UA). Để tính MetaF(UA), DataF được đưa qua các bộ học cơ sở trong UA. Việc chạy các bộ học cơ sở được chia thành ba nhóm:

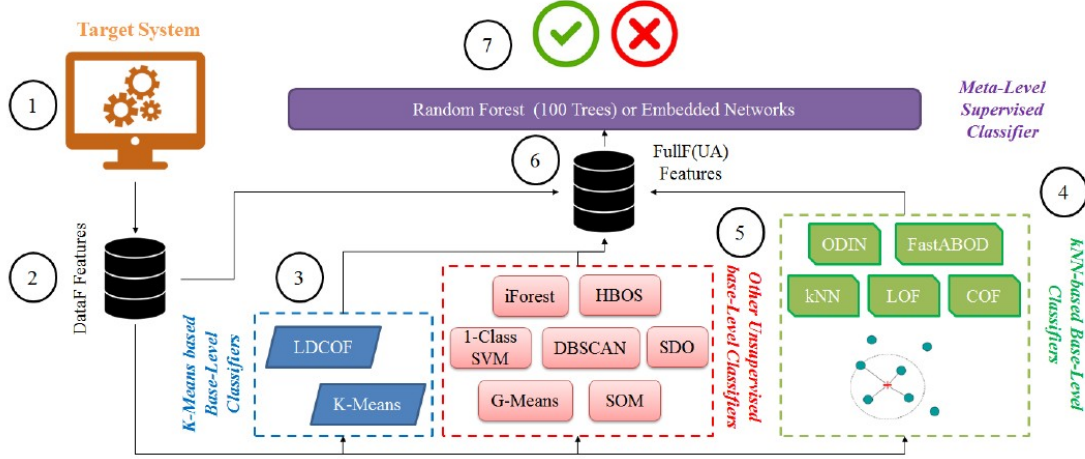
- **Nhóm dựa trên K-Means, bước 3:** K-Means gán num và bin sau khi tìm cụm gần nhất. Thông tin này góp phần tạo điểm LDCOF, vì vậy LDCOF có thể được tính như phần mở rộng của kết quả K-Means.
- **Nhóm dựa trên kNN, bước 4:** ODIN, FastABOD, kNN, LOF và COF đều cần tìm k láng giềng gần nhất. Việc tìm kiếm chỉ cần thực hiện một lần, sau đó kết quả được chuyển cho từng thuật toán để nhanh chóng tính đặc trưng num và bin.
- **Các bộ học không giám sát khác, bước 5:** những thuật toán này dùng các heuristic không chồng lấn nên phải chạy độc lập.

Các đặc trưng ua.num, ua.bin sinh ở bước 3-5 cùng DataF tạo FullF(UA) tại bước 6. Tập này được đưa vào bộ phân loại có giám sát tầng meta - Random Forest hoặc mạng có embedding - để quyết định bất thường ở bước 7, hoàn tất hiện thực Stacker(UA).

parameters' values. In addition, the table reports on # Best, the

neighbours to a given data point and then incrementally

Figure 6: Instantiation of the proposed Stacker. The figure identifies 7 steps ① to ⑦, described in Section 7.



Hình 6

**Hình 6.** Hiện thực Stacker được đề xuất với bảy bước: hệ thống đích; tạo DataF; nhóm K-Means; nhóm kNN; các bộ học không giám sát khác; tạo FullF(UA); và bộ phân loại tầng meta đưa ra quyết định bình thường/bất thường.

## 7.2. Hiện thực Stacker

Việc nhóm bộ học cơ sở thành các nhóm ở bước 3, 4 và 5 giúp đơn giản hóa kiến trúc và tránh lặp lại thao tác, đặc biệt là tìm kiếm kNN. Tuy nhiên, ngay cả sau khi nhóm, hệ thống vẫn phải hiện thực nhiều thuật toán không giám sát. Phần lớn là thuật toán phổ biến và có hiện thực công khai bằng nhiều ngôn ngữ, phù hợp với các môi trường khác nhau. Phần cuối bài báo nêu hướng giảm số lượng bộ học cơ sở so với toàn bộ UA.

## 7.3. Độ phức tạp tính toán

Stacker phát hiện xâm nhập bằng cách chạy các bộ học cơ sở rồi chạy bộ học tầng meta. Các bộ học cơ sở có thể thực thi song song, nhưng tầng meta phải chạy sau chúng. Với UA gồm 14 thuật toán, Random Forest làm tầng meta và cc là hàm độ phức tạp:

$$cc(Stacker(FullF(UA))) = \max\{cc(ua) \mid ua \in UA\} + cc(RandomForest) \\ \approx cc(FastABOD) + cc(RandomForest)$$

Theo Bảng 2, FastABOD có độ phức tạp tiệm cận cao nhất ở cả huấn luyện và kiểm thử. Độ phức tạp của Random Forest tương tự iForest vì huấn luyện và kiểm thử cây quyết định với cây cô lập có cùng bậc tiệm cận. Với  $n$  là kích thước tập huấn luyện:

$$Huấn\ luyện: cc(Stacker(UA)) \approx O(n^2 + nk^2) + O(ts \log s) \in O(n^2)$$

$$Kiểm\ thử: cc(Stacker(UA)) \approx O(k^2 \log n) + O(t \log s) \in O(\log n)$$

Ngoài  $n$ , các giá trị  $k$ ,  $t$ ,  $s$  là tham số hằng của thuật toán.  $k$  được chọn bằng grid search khi huấn luyện FastABOD;  $t$  là số cây của Random Forest, được đặt bằng 100 theo kết quả Mục 6;  $s$  là kích thước mẫu dùng để dựng cây. Ngay cả khi  $s = n$ , độ phức tạp tiệm cận của Stacker vẫn tương tự nhiều bộ phân loại khác: bậc hai khi huấn luyện và logarit khi kiểm thử.

## 8. SO SÁNH VỚI CÁC NGHIÊN CỨU VỀ PHÁT HIỆN XÂM NHẬP

Cuối cùng, công trình được đặt trong tương quan với các nghiên cứu đã dùng cùng các tập dữ liệu công khai. Các biến thể zero-day mới tạo không được so sánh vì chưa xuất hiện trong công trình trước. Các bài báo so sánh được chọn theo tiêu chí:

- Mới, tức công bố trong năm năm gần nhất, để bảo đảm thuật toán dùng các chiến lược phân loại hiện đại.
- Ít nhất hai nghiên cứu cho mỗi tập dữ liệu trong bảy tập, tạo đường cơ sở tối thiểu.
- Ưu tiên bài báo tạp chí quốc tế hơn bài hội nghị khi có thể.

Tổng cộng có 15 bài: 2 cho ADFANet [87], [88]; 3 cho CICIDS17 [80], [82], [83]; 3 cho CIDDS-001 [82], [84], [89]; 2 cho ISCX12 [79], [81]; 3 cho NSL-KDD [78], [81], [83]; 3 cho UGR16 [76], [85], [86]; và 4 cho UNSW-NB15 [76], [77], [81], [83]. Vì các công trình này dùng những chỉ số khác MCC, nhóm tác giả tính thêm Accuracy, Precision, Recall, F1, AUC và FPR của Stacker trên từng tập dữ liệu.

**Bảng 6. So sánh Stacker với các nghiên cứu ML phát hiện xâm nhập trên cùng tập dữ liệu**

| Nghiên cứu     | Tập dữ liệu      | ACC           | Precision    | Recall       | F1           | AUC          | FPR          |
|----------------|------------------|---------------|--------------|--------------|--------------|--------------|--------------|
| [87]           | ADFAvNet         | 0,9759        | -            | 1,000        | 0,983        | -            | -            |
| [88]           | ADFAvNet         | 0,9530        | 0,995        | 0,958        | -            | -            | -            |
| <b>Stacker</b> | <b>ADFAvNet</b>  | <b>0,9956</b> | <b>0,998</b> | <b>0,988</b> | <b>0,993</b> | <b>0,993</b> | <b>0,012</b> |
| [80]           | CICIDS17         | 0,9988        | -            | -            | -            | -            | 0,002        |
| [83]           | CICIDS17         | 0,9900        | -            | -            | -            | 1,000        | -            |
| [82]           | CICIDS17         | -             | -            | -            | 0,787        | 0,781        | -            |
| <b>Stacker</b> | <b>CICIDS17</b>  | <b>0,9997</b> | <b>0,998</b> | <b>1,000</b> | <b>0,999</b> | <b>0,999</b> | <b>0,000</b> |
| [89]           | CIDDS-001        | 0,9954        | -            | -            | -            | 0,969        | -            |
| [84]           | CIDDS-001        | 0,9705        | -            | -            | -            | -            | 0,021        |
| [82]           | CIDDS-001        | -             | -            | -            | 0,675        | 0,720        | -            |
| <b>Stacker</b> | <b>CIDDS-001</b> | <b>0,9981</b> | <b>0,998</b> | <b>0,998</b> | <b>0,998</b> | <b>0,998</b> | <b>0,002</b> |
| [79]           | ISCX12           | 0,9992        | -            | 0,989        | -            | -            | 0,038        |
| [81]           | ISCX12           | 0,9988        | -            | -            | -            | -            | 0,035        |
| <b>Stacker</b> | <b>ISCX12</b>    | <b>0,9880</b> | <b>0,903</b> | <b>0,943</b> | <b>0,923</b> | <b>0,923</b> | <b>0,057</b> |
| [78]           | NSL-KDD          | 0,9899        | -            | 0,996        | -            | -            | 0,056        |
| [81]           | NSL-KDD          | 0,9823        | -            | -            | -            | -            | 0,033        |
| [83]           | NSL-KDD          | 0,9750        | 0,976        | 0,984        | -            | -            | 0,075        |
| <b>Stacker</b> | <b>NSL-KDD</b>   | <b>0,9939</b> | <b>0,997</b> | <b>0,990</b> | <b>0,993</b> | <b>0,993</b> | <b>0,010</b> |
| [85]           | UGR16            | 0,9961        | 0,996        | -            | -            | -            | -            |
| [76]           | UGR16            | 0,9719        | 0,988        | 0,981        | -            | -            | -            |
| [86]           | UGR16            | -             | 0,897        | 0,887        | 0,888        | 0,868        | -            |
| <b>Stacker</b> | <b>UGR16</b>     | <b>0,9967</b> | <b>0,975</b> | <b>0,999</b> | <b>0,987</b> | <b>0,987</b> | <b>0,001</b> |
| [77]           | UNSW-NB15        | 0,9613        | -            | -            | -            | -            | -            |
| [76]           | UNSW-NB15        | 0,9400        | 0,960        | 0,930        | 0,950        | -            | -            |
| [81]           | UNSW-NB15        | 0,9371        | -            | -            | -            | -            | 0,046        |
| [83]           | UNSW-NB15        | 0,8800        | -            | -            | -            | 0,860        | -            |
| <b>Stacker</b> | <b>UNSW-NB15</b> | <b>0,9719</b> | <b>0,982</b> | <b>0,946</b> | <b>0,964</b> | <b>0,964</b> | <b>0,054</b> |

Bảng được nhóm theo tập dữ liệu; dòng Stacker là kết quả của bài báo. Ô trống xuất hiện khi nghiên cứu tương ứng không báo cáo chỉ số đó. Các chỉ số được chọn là những chỉ số phổ biến nhất trong các bài so sánh và tạo đường cơ sở phù hợp.

Accuracy được hầu hết công trình báo cáo và là tiêu chí so sánh đầu tiên. Trừ ISCX12, Stacker đạt Accuracy cao hơn trên mọi tập dữ liệu, tức giảm cả FP lẫn FN. Điều này đáng chú ý vì nghiên cứu không thực hiện tối ưu riêng cho từng tập dữ liệu, như thiết kế thủ công đặc trưng riêng hoặc tiền xử lý chuyên biệt. Do đó, Stacker có phạm vi áp dụng rộng.

Với ISCX12, Stacker thấp hơn nhẹ so với [79], [81]. Cả hai công trình đó đều phân tích cụm để đặc trưng hóa tốt hơn các loại tấn công; đây là bước đầu tạo lợi thế lớn cho điểm số. Ngoài ra, [79] dùng tập huấn luyện lớn hơn và tạo thêm 30 đặc trưng riêng cho tập dữ liệu, còn [81] kết hợp Extra Trees với cửa sổ thời gian.

Nhìn chung, Stacker có khả năng cạnh tranh cao với các nghiên cứu gần đây, cải thiện kết quả trên 6 trong 7 tập dữ liệu và đồng thời được thiết kế vững hơn trước các cuộc tấn công chưa biết - một thách thức lớn đối với cả học thuật lẫn công nghiệp.

## 9. KẾT LUẬN VÀ BÀI HỌC RÚT RA

Bài báo lý giải nhu cầu kết hợp thuật toán học máy có giám sát và không giám sát để bộ phát hiện xâm nhập mạng xử lý được cả mối đe dọa đã biết lẫn chưa biết/zero-day. Các tác giả phân tích những thuật toán hai nhóm và các cách học meta cho phép phối hợp nhiều bộ phân loại. Quá trình này dẫn tới một tổ hợp Stacking hai tầng, điều phối các bộ học không giám sát ở tầng cơ sở và một bộ phân loại có giám sát ở tầng meta.

So với bộ phát hiện xâm nhập có giám sát truyền thống, Stacker giảm khoảng 33% số trường hợp phân loại sai và cải thiện rõ rệt phát hiện zero-day, thể hiện qua Recall trên tấn công chưa biết. Vì vậy, nó có tiềm năng trở thành một giải pháp thống nhất cho phát hiện xâm nhập. Hiệu quả được xác nhận bằng so sánh rộng với các công trình gần đây trên cùng tập dữ liệu công khai; Stacker vượt các nghiên cứu trước trên 6 trong 7 tập dữ liệu.

Hướng nghiên cứu hiện tại và tương lai tập trung giảm số bộ học cơ sở không giám sát, nhằm giúp Stacker dễ hiện thực hơn nhưng không làm giảm chỉ số. Lập luận chính là mức đóng góp của từng bộ học vào quyết định cuối không giống nhau. Nhóm tác giả dự kiến thực hiện phân tích độ nhạy quy mô lớn để xác định tập bộ học cơ sở tối thiểu nhưng vẫn giữ khả năng phát hiện xuất sắc. Họ cũng nghiên cứu tương quan giữa điểm số của các thuật toán không giám sát để định lượng tính hiệp lực và đa dạng, thay vì chỉ lựa chọn định tính theo họ thuật toán.

# TÀI LIỆU THAM KHẢO

*Danh mục tài liệu tham khảo được giữ nguyên ngôn ngữ gốc để bảo toàn chính xác tên tác giả, tên công trình, tạp chí, hội nghị, DOI và liên kết.*

- [1] Chandola, V., Banerjee, A., Kumar, V. “Anomaly detection: A survey”. (2009) ACM computing surveys (CSUR), 41(3), 15.
- [2] Mishra, P., Pilli, E. S., Varadharajan, V., & Tupakula, U. (2017). Intrusion detection techniques in cloud environment: A survey. Journal of Network and Computer Applications, 77, 18-47.
- [3] Zhao, Z., Cerf, S., Birke, R., Robu, B., Bouchenak, S., Mokhtar, S. B., & Chen, L. Y. (2019, June). Robust anomaly detection on unreliable data. In 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (pp. 630637). IEEE.
- [4] Palmieri, F. (2019). Network anomaly detection based on logistic regression of nonlinear chaotic invariants. Journal of Network and Computer Applications, 148, 102460.
- [5] Garcia, N., Alcaniz, T., González-Vidal, A., Bernabe, J. B., Rivera, D., & Skarmeta, A. Distributed real-time SlowDoS attacks detection over encrypted traffic using Artificial Intelligence. Journal of Network and Computer Applications, 173, 102871.
- [6] Bilge, L., & Dumitraş, T. (2012, October). Before we knew it: an empirical study of zero-day attacks in the real world. In Proceedings of the 2012 ACM conference on Computer and communications security (pp. 833-844). ACM.
- [7] Check Point Research, (2020), <https://research.checkpoint.com/2020/the-2020-cyber-securityreport/> (online)
- [8] ENISA (2020), Threat Landscape Report, <https://www.enisa.europa.eu/news/enisa-news/enisa-threatlandscape-2020> (online).
- [9] Malformed Header Attack (online), [https://support.ixiacom.com/strikes/generic/ixia/spamassassin\\_malformed\\_email\\_header\\_denial\\_of\\_service\\_attack.xml](https://support.ixiacom.com/strikes/generic/ixia/spamassassin_malformed_email_header_denial_of_service_attack.xml)
- [10] Berrueta, E., Morato, D., Magaña, E., & Izal, M. (2019). A survey on detection techniques for cryptographic ransomware. IEEE Access, 7, 144925-144944.
- [11] Committee on National Security Systems - CNSSI No. 4009 “Committee on National Security Systems (CNSS) Glossary”, April 2015.
- [12] Carcillo, F., Le Borgne, Y. A., Caelen, O., Kessaci, Y., Oblé, F., & Bontempi, G. (2019). Combining unsupervised and supervised learning in credit card fraud detection. Information Sciences.
- [13] Arora, Anshul, et al. “Hybrid Android malware detection by combining supervised and unsupervised learning.” Proceedings of the 24th Annual International Conference on Mobile Computing and Networking. 2018.
- [14] Comar, Prakash Mandayam, et al. “Combining supervised and unsupervised learning for zero-day malware detection.” 2013 Proceedings IEEE INFOCOM. IEEE, 2013.
- [15] Yamanishi, Kenji, and Jun-ichi Takeuchi. “Discovering outlier filtering rules from unlabeled data: combining a supervised learner with an unsupervised learner.” Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining. 2001.
- [16] Ren, Haoyu, et al. “SUW-Learn: Joint Supervised, Unsupervised, Weakly Supervised Deep Learning for Monocular Depth Estimation.” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops. 2020.
- [17] Li, Zhipeng, et al. “SUPER Learning: A Supervised-Unsupervised Framework for Low-Dose CT Image Reconstruction.” Proceedings of the IEEE International Conference on Computer Vision Workshops. 2019.
- [18] Benvenuto, Federico, et al. “A hybrid supervised/unsupervised machine learning approach to solar flare prediction.” The Astrophysical Journal 853.1 (2018): 90.
- [19] Le, Lei, Andrew Patterson, and Martha White. “Supervised autoencoders: Improving generalization performance with unsupervised regularizers.” Advances in Neural Information Processing Systems. 2018.
- [20] Vanschoren J (2010) Understanding machine learning performance with experiment databases. PhD thesis, Arenberg Doctoral School of Science, Engineering & Technology, Katholieke Universiteit Leuven.
- [21] Breiman, L.. Bagging predictors. Machine Learning, 26(2):123– 140, 1996.
- [22] Schapire, R.E. The strength of weak learnability. Mach Learn 5, 197–227 (1990). <https://doi.org/10.1007/BF00116037>
- [23] Wolpert, David. (1992). Stacked Generalization. Neural Networks. 5. 241-259. 10.1016/S0893-6080(05)80023-1.

- [24] E. Alpaydin and C. Kaynak. Cascading classifiers. *Kybernetika*, 34:369–374, 1998.
- [25] J. Gama and Brazdil P. Cascade generalization. *Machine Learning*, 41(3):315–343, 2000.
- [26] C. Ferri, P. Flach, and J. Hernandez-Orallo. Delegating classifiers. In *Proceedings of the Twenty-first International Conference on Machine Learning, (ICML’04)*, pages 289–296, 2004.
- [27] Ortega, Julio & Koppel, Moshe & Argamon, Shlomo. (2001). Arbitrating Among Competing Classifiers Using Learned Referees. *Knowledge and Information Systems*. 3. 470-490. 10.1007/PL00011679.
- [28] Zoppi, T. et. al., Meta-Learning to Improve Unsupervised Intrusion Detection in Cyber-Physical Systems, under review at *ACM Transactions on Cyber-Physical Systems*, 2021 – Uploaded for review at: (online) <https://drive.google.com/file/d/1ZHmn7bBu56gDZ6UTJ4zqIwcyZwLnA4hP/view?usp=sharing>.
- [29] Goldstein, M., & Uchida, S. (2016). A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PloS one*, 11(4), e0152173.
- [30] Zoppi, Tommaso, Andrea Ceccarelli, Tommaso Capecchi, and Andrea Bondavalli. “Unsupervised Anomaly Detectors to Detect Intrusions in the Current Threat Landscape.” *arXiv preprint arXiv:2012.11354* (2020) – to appear on the *ACM Transactions on Data Science*.
- [31] Breiman, Leo. “Random forests.” *Machine learning* 45.1 (2001): 532.
- [32] Rätsch, Gunnar, Takashi Onoda, and K-R. Müller. “Soft margins for AdaBoost.” *Machine learning* 42.3 (2001): 287-320.
- [33] Hearst, Marti A., et al. “Support vector machines.” *IEEE Intelligent Systems and their applications* 13.4 (1998): 18-28.
- [34] Wu, Z., Wang, J., Hu, L., Zhang, Z., & Wu, H. (2020). A network intrusion detection method based on semantic Re-encoding and deep learning. *Journal of Network and Computer Applications*, 102688.
- [35] Gamage, S., & Samarabandu, J. (2020). Deep learning methods in network intrusion detection: A survey and an objective comparison. *Journal of Network and Computer Applications*, 169, 102767.
- [36] Chen, Liming, and Algirdas Avizienis. “N-version programming: A fault-tolerance approach to reliability of software operation.” *Proc. 8th IEEE Int. Symp. on Fault-Tolerant Computing (FTCS-8)*. Vol. 1. 1978.
- [37] Siewiorek, D. P. (1975). Reliability modeling of compensating module failures in majority voted redundancy. *IEEE Transactions on Computers*, 100(5), 525-533.
- [38] Gouveia, A., & Correia, M. (2020). Deep Learning for Network Intrusion Detection: An Empirical Assessment. *Recent Advances in Security, Privacy, and Trust for Internet of Things (IoT) and CyberPhysical Systems (CPS)*, 191.
- [39] Liao, Yihua, and V. Rao Vemuri. “Use of k-nearest neighbor classifier for intrusion detection.” *Computers & security* 21.5 (2002): 439-448.
- [40] Srivastava, S., Gupta, M. R., & Frigyik, B. A. (2007). Bayesian quadratic discriminant analysis. *Journal of Machine Learning Research*, 8(Jun), 1277-1305.
- [41] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*.
- [42] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaee, and Ali A Ghorbani. 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *computers & security* 31, 3 (2012), 357–374.
- [43] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A Ghorbani. 2009. A detailed analysis of the KDD CUP 99 data set. In *Computational Intelligence for Security and Defense Applications, 2009. CISDA 2009. IEEE Symposium on*. IEEE, 1–6.
- [44] Ring, M., Wunderlich, S., Grödl, D., Landes, D., & Hotho, A. (2017, June). Flow-based benchmark data sets for intrusion detection. In *Proceedings of the 16th European Conference on Cyber Warfare and Security*. ACPI (pp. 361-369).
- [45] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *Military Communications and Information Systems Conference (MilCIS), 2015*. IEEE, 1–6.
- [46] Kriegel H-P, Zimek A. “Angle-based outlier detection in highdimensional data”. *Proc. of the 14th ACM SIGKDD Int. Conf. on Knowledge discovery data mining; ’08*. p. 444–452.
- [47] Chicco, Davide, and Giuseppe Jurman. “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation.” *BMC genomics* 21.1 (2020): 6.
- [48] Boughorbel, Sabri, Fethi Jarray, and Mohammed El-Anbari. “Optimal classifier for imbalanced data using Matthews Correlation Coefficient metric.” *PloS one* 12.6 (2017): e0177678.

- [49] G. O. Campos, A. Zimek, J. Sander, R. J. Campello, B. Micenkova, E. Schubert, I. Assent, and M. E. Houle, "On the evaluation of outlier detection: Measures, datasets, and an empirical study", in *Lernen, Wissen, Daten, Analysen 2016. CEUR work-shop proceedings*, 2016.
- [50] Maciá-Fernández, G., Camacho, J., Magán-Carrión, R., GarcíaTeodoro, P., & Theron, R. (2018). UGR '16: A new dataset for the evaluation of cyclostationarity-based network IDSs. *Computers & Security*, 73, 411-424.
- [51] Haider, W., Hu, J., Slay, J., Turnbull, B. P., & Xie, Y. (2017). Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. *Journal of Network and Computer Applications*, 87, 185-192.
- [52] Zoppi, T., Ceccarelli, A., & Bondavalli, A. (2019, October). "Evaluation of Anomaly Detection algorithms made easy with RELOAD" In *Proceedings of the 30th Int. Symposium on Soft-ware Reliability Engineering (ISSRE)*, pp 446-455, IEEE.
- [53] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, January 2018
- [54] Azhagusundari, B., and Antony Selvadoss Thanamani. "Feature selection based on information gain." *International Journal of Innovative Technology and Exploring Engineering (IJITEE)* 2.2 (2013): 18-21.
- [55] Chan, P. K., & Stolfo, S. J. (1995). A comparative evaluation of voting and meta-learning on partitioned data. In *Machine Learning Proceedings 1995* (pp. 90-98). Morgan Kaufmann.
- [56] Avizienis, A., Laprie, J. C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE transactions on dependable and secure computing*, 1(1), 1133.
- [57] Wan, X., Wang, W., Liu, J., & Tong, T. (2014). Estimating the sample mean and standard deviation from the sample size, median, range and/or interquartile range. *BMC medical research methodology*, 14(1), 135.
- [58] Guo, Cheng, and Felix Berkhahn. "Entity embeddings of categorical variables." *arXiv preprint arXiv:1604.06737* (2016).
- [59] B. Lau, The Right Way to Use Deep Learning for Tabular Data | Entity Embedding, <https://towardsdatascience.com/the-right-way-to-use-deep-learning-for-tabular-data-entity-embeddingb5c4aaf1423a>
- [60] Goldstein, Markus, and Andreas Dengel. "Histogram-based outlier score (hbos): A fast unsupervised anomaly detection algorithm." *KI-2012: Poster / Demo Track* (2012): 59-63.
- [61] Hautamaki, V., Karkkainen, I., & Franti, P. (2004, August). Outlier detection using k-nearest neighbour graph. *Pattern Recognition. ICPR 2004. Proceedings of the 17th Int. Conf on (Vol. 3, pp. 430433)*. IEEE.
- [62] M. Amer, M. Goldstein, and S. Abdennadher, "Enhancing one-class support vector machines for unsupervised anomaly detection," in *Proceedings of the ACM SIGKDD Workshop on Outlier Detection and Description*. ACM, 2013, pp. 8–15.
- [63] Vázquez, Félix Iglesias, Tanja Zseby, and Arthur Zimek. "Outlier detection based on low density models." 2018 IEEE Int. Conference on Data Mining Workshops (ICDMW). IEEE, 2018.
- [64] Kohonen, T. (1997, June). Exploration of very large databases by self-organizing maps. In *Proc. of Int. Conference on Neural Networks (ICNN'97) (Vol. 1, pp. PL1-PL6)*. IEEE.
- [65] Hamerly, G., & Elkan, C. (2004). Learning the k in k-means. In *Advances in neural information processing systems* (pp. 281-288).
- [66] Mennatallah Amer and Markus Goldstein. 2012. Nearest-neighbor and clustering based anomaly detection algorithms for rapidminer. In *Proceedings of the 3rd RapidMiner Community Meeting and Conference (RCOMM 2012)*.
- [67] Jian Tang, Zhixiang Chen, Ada Wai-Chee Fu, and David W Cheung. 2002. Enhancing effectiveness of outlier detections for low density patterns. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 535–548.
- [68] Schubert, E., & Gertz, M. (2017, October). Intrinsic t-stochastic neighbor embedding for visualization and outlier detection. In *Int. Conference on Similarity Search and Applications* (pp. 188-203). Springer, Cham.
- [69] Martin Ester, Han-peter Kriegel, Jorg Sander, Xiaowei Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise", 2nd Int. Conference on Knowledge Discovery and Data Mining (KDD-96)
- [70] Breunig, M. M., Kriegel, H. P., Ng, R. T., & Sander, J. (2000, May). LOF: identifying density-based local outliers. In *ACM sigmod record (Vol. 29, No. 2, pp. 93-104)*. ACM.
- [71] Radovanović, M., Nanopoulos, A., & Ivanović, M. (2014). Reverse nearest neighbors in unsupervised distance-based outlier detection. *IEEE transactions on knowledge and data engineering*, 27(5), 13691382.
- [72] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008, December). Isolation forest. In *2008 8th Int. Conference on Data Mining* (pp. 413-422). IEEE.

- [73] Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A kmeans clustering algorithm. *Journal of the Royal Statistical Society. Series C*, 28(1), 100-108.
- [74] Brazdil P, Giraud-Carrier C, Soares C, Vilalta R (2009) *Metalearning: applications to data mining*. Springer, Berlin.
- [75] Lemke, Christiane & Budka, Marcin & Gabrys, Bogdan. (2013). *Metalearning: a survey of trends and technologies*. *Artificial Intelligence Review*. DOI: 10.1007/s10462-013-9406-y. 10.1007/s10462-013-9406-y.
- [76] Rajagopal, S., Kundapur, P. P., & Hareesha, K. S. (2020). A stacking ensemble for network intrusion detection using heterogeneous datasets. *Security and Communication Networks*, 2020.
- [77] Sharma, J., Giri, C., Granmo, O. C., & Goodwin, M. (2019). Multilayer intrusion detection system with ExtraTrees feature selection, extreme learning machine ensemble, and softmax aggregation. *EURASIP Journal on Information Security*, 2019(1), 15.
- [78] Parsaei, M. R., Rostami, S. M., & Javidan, R. (2016). A hybrid data mining approach for intrusion detection on imbalanced NSL-KDD dataset. *International Journal of Advanced Computer Science and Applications*, 7(6), 20-25.
- [79] Soheily-Khah, S., Marteau, P. F., & Béchet, N. (2018, April). Intrusion detection in network systems through hybrid supervised and unsupervised machine learning process: A case study on the iscx dataset. In *2018 1st International Conference on Data Intelligence and Security (ICDIS)* (pp. 219-226). IEEE.
- [80] Stiawan, D., Idris, M. Y. B., Bamhdi, A. M., & Budiarto, R. (2020). CICIDS-2017 Dataset Feature Analysis With Information Gain for Anomaly Detection. *IEEE Access*, 8, 132911-132921.
- [81] Idhammad, M., Afdel, K., & Belouch, M. (2018). Semi-supervised machine learning approach for DDoS detection. *Applied Intelligence*, 48(10), 3193-3208.
- [82] Pontes, C., Souza, M., Gondim, J., Bishop, M., & Marotta, M. (2019). A new method for flow-based network intrusion detection using the inverse Potts model. *arXiv preprint arXiv:1910.07266*.
- [83] Chand, N., Mishra, P., Krishna, C. R., Pilli, E. S., & Govil, M. C. (2016, April). A comparative analysis of SVM and its stacking with other classification algorithm for intrusion detection. In *2016 International Conference on Advances in Computing, Communication, & Automation (ICACCA)*(Spring) (pp. 1-6). IEEE.
- [84] Idhammad, M., Afdel, K., & Belouch, M. (2018). Distributed intrusion detection system for cloud environments based on data mining techniques. *Procedia Computer Science*, 127, 35-41.
- [85] Larriva-Novo, X., Vega-Barbas, M., Villagrà, V. A., Rivera, D., Álvarez-Campana, M., & Berrocal, J. (2020). Efficient Distributed Preprocessing Model for Machine Learning-Based Anomaly Detection over Large-Scale Cybersecurity Datasets. *Applied Sciences*, 10(10), 3430.
- [86] Magán-Carrión, R., Urda, D., Díaz-Cano, I., & Dorronsoro, B. (2020). Towards a Reliable Comparison and Evaluation of Network Intrusion Detection Systems Based on Machine Learning Approaches. *Applied Sciences*, 10(5), 1775.
- [87] Mouttaqi, T., Rachidi, T., & Assem, N. (2017, September). Reevaluation of combined Markov-Bayes models for host intrusion detection on the ADFA dataset. In *2017 Intelligent Systems Conference (IntelliSys)* (pp. 1044-1052). IEEE.
- [88] Sharma, R., & Chaurasia, S. (2018). An enhanced approach to fuzzy C-means clustering for anomaly detection. In *Proceedings of First International Conference on Smart System, Innovations and Computing* (pp. 623-636). Springer, Singapore.
- [89] Idhammad, M., Afdel, K., & Belouch, M. (2018). Detection system of HTTP DDoS attacks in a cloud environment based on information theoretic entropy and random forest. *Security and Communication Networks*, 2018.
- [90] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. *Cybersecurity*, 2(1), 20
- [91] Lee, C., & Lee, G. G. (2006). Information gain and divergencebased feature selection for machine learning-based text categorization. *Information processing & management*, 42(1), 155165.
- [92] Obenshain, Daniel, Thomas Tantillo, Amy Babay, John Schultz, Andrew Newell, Md Edadul Hoque, Yair Amir, and Cristina NitaRotaru. "Practical intrusion-tolerant networks." In *2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS)*, pp. 45-56. IEEE, 2016.
- [93] Python Code for Running Embedded Networks (online): [https://drive.google.com/drive/folders/1ZYnywuzGLAJLpEWgBrwU7a9ekGYq4u\\_D?usp=sharing](https://drive.google.com/drive/folders/1ZYnywuzGLAJLpEWgBrwU7a9ekGYq4u_D?usp=sharing)
- [94] Support Files for Analysis (online): [https://drive.google.com/file/d/1ZJWBwpbq3VOfdK37pd6EjxUQp\\_3\\_yzMSm/view?usp=sharing](https://drive.google.com/file/d/1ZJWBwpbq3VOfdK37pd6EjxUQp_3_yzMSm/view?usp=sharing)
- [95] Bessani, A., Correia, M., Quaresma, B., André, F., & Sousa, P. (2013). DepSky: dependable and secure storage in a cloud-ofclouds. *Acm transactions on storage (tos)*, 9(4), 1-33.